

애드지니 프로젝트 통합 보고서

AdGenie — 소상공인을 위한 광고 문구 · 포스터 자동 생성 서비스

예쁜 결과물을 만드는 서비스는 많습니다.
소상공인이 실제로 겪는 위험은 “몰라서 쓴 한 문장”입니다.
애드지니는 **법적으로 안전한** 광고를 만듭니다.

작성 기준일 2026-08-29

대상 기간 2026-08-04 ~ 2026-08-30

팀 구성
김도혁 문구 모델, 챗봇 기능 구현
김소원 프론트엔드, 백엔드
정진우 PM (서빙 인프라 보조)
하태진 서빙 인프라
황지우 포스터 생성 모델, 후처리

목차

1. 프로젝트 개요	3
2. 배경 및 문제 정의	5
3. 서비스 소개	7
4. 시스템 설계와 기술 선정	11
5. 기술 구현	15
6. 개발 과정과 의사결정	28
7. 결과 및 평가	32
8. 문제 해결 과정	34
9. 한계점 및 향후 계획	37
10. 프로젝트 회고	39
11. 참고자료	41

1. 프로젝트 개요

1-1. 프로젝트 정보

항목	내용
프로젝트명	애드지니 (AdGenie)
한 줄 소개	소상공인이 광고 문구와 포스터를 만들 때, 법을 몰라서 쓰게 되는 한 문장을 사전에 걸러내는 생성 서비스
팀명	Final-Project-team5
프로젝트 기간	2026-08-04 ~ 2026-08-30
저장소	github.com/Final-Project-team5/Final-Project-team5
산출물	문구 모델 API · 포스터 모델 API · React 웹 프론트엔드 · GCP VM 배포
작성 기준일	2026-08-29

광고 문구를 만들어 주는 도구는 이미 많음. 애드지니가 다른 지점은 결과물의 완성도가 아니라 **결과물이 법적으로 안전한가**를 서비스의 1차 기능으로 올려놓은 것임. 생성은 수단이고, 검증이 핵심임.

1-2. 팀 구성과 역할

담당	파트	주요 산출물
김도혁	문구 모델 · 규제 검증 · 챗봇	규제 룰 사전 50개, 골드셋 127건, 하이브리드 검증 레이어, 챗봇 슬롯필링
김소원	프론트엔드 · 백엔드 연동	화면 7종, mock/real 이중 어댑터, UI/UX 스펙 문서
정진우	PM (서빙 인프라 보조)	범위 확정, 협업 체계 구축, 코드 리뷰, VM 구축·배포, 폰트 검증
하태진	서빙 인프라	Cloud Run 콜드 스타트 분해, 모델 저장 위치 비교 조사
황지우	포스터 생성 모델 · 후처리	2단계 파이프라인, 비정사각 지원, 서비스형 레이아웃, Design Planner

1-3. 개발 일정

날짜	마일스톤
8/4	저장소·협업 일지 템플릿·프로젝트 범위 확정
8/10	mock API 배포, 파트 간 인터페이스 합의
8/13	GCP VM 외부 노출, 프론트 실연동 착수
8/17	중간 점검 — 전원 시연
8/28	모델 코드 동결
8/25~29	실서버 E2E 진행 후 오류 수정 · 최종 시연 UI 정리
8/30	최종 제출

코드 동결 이후는 신규 기능을 만드는 구간이 아니라 **실서버에서만 드러나는 문제를 잡는 구간**이었음. 이 기간 작업의 대부분이 버그 수정이었고, 개발 환경이나 로컬에서는 재현되지 않던 것들임.

2. 배경 및 문제 정의

2-1. 배경

푸드·뷰티·건강 관련 업종은 광고 규제가 촘촘함. 표시광고법을 기본으로 식품표시광고법, 건강기능식품법, 화장품법, 약사법, 의료기기법이 겹쳐 적용되고, 학원·체육관 같은 서비스업에는 학원법 시행령과 방문판매법, 국민체육진흥법이 걸림.

문제는 이 규제를 확인할 사람이 소상공인 쪽에는 없다는 것임. 법무 인력을 두는 규모가 아니고, 광고 대행을 쓸 여력도 없는 경우가 많음. 그래서 규제 위반은 악의가 아니라 **몰라서** 발생함.

"아토피 치료되는 크림"	→	화장품법: 질환명 언급은 의약품 오인 광고
"수강료 환불 불가"	→	학원법 시행령: 수강료 반환 기준 위반
"고혈압 낮춰주는 팔찌"	→	약사법 / 의료기기법: 의료기기 오인

여기에 생성형 AI가 들어오면서 위험이 오히려 커짐. **LLM이 매력적인 카피를 잘 만든다는 사실 자체가 위험 요인**임. 매력적인 광고 표현은 대개 최상급·효능 단정·확정 표현이고, 그것이 곧 규제 대상이기 때문임. 도구가 좋아질수록 사용자는 더 빠르게, 더 자연스럽게 위반 문구를 얻게 됨.

2-2. AS-IS → 문제 → TO-BE

AS-IS — 현재 소상공인의 광고 제작 과정

- ① 문구를 직접 쓰거나 생성형 AI로 뽑는다
↓
- ② 이 표현이 문제가 되는지 확인할 방법이 없음
(법령 원문을 찾아야 하고, 어떤 법이 적용되는지부터 모른다)
↓
- ③ 디자인 도구로 포스터를 만든다
↓
- ④ 게시함 → 문제가 되면 그때 알게 됨

문제 — 세 가지로 정리함

#	문제	구체적인 형태
P1	위반 여부를 사전에 알 수 없음	사후 적발 구조. 게시 전에 확인할 수단이 없음
P2	생성형 AI가 위반 표현을 더 잘 만든다	매력적인 카피 ≙ 최상급·효능 단정 = 규제 대상
P3	문구와 이미지가 분리된 도구에 있음	문구 도구와 디자인 도구를 오가며 수작업 합성

TO-BE — 애드지니가 만드는 상태

- ① 챗봇이 업종·용도·강조점을 순차로 수집함
↓
- ② 문구 생성과 동시에 규제 검증이 붙어 나온다
(block / warn 등급 + 근거 법령 + 대체 표현)
↓
- ③ 검증을 통과한 문구가 그대로 포스터에 합성됨
↓
- ④ AI 생성물 고지까지 붙어 다운로드됨

핵심은 ②와 ③ 사임. **포스터에 실리는 문구는 규제 검증을 거친 문구뿐**이라는 것이 서비스의 핵심임.

2-3. 해결 목표

문제	목표	구현 방식
P1	게시 전 검증	룰 기반 자동 검사 + LLM 2차 재검증 하이브리드
P2	생성 단계에서 억제	프롬프트 금지 규칙 + 생성 직후 자동 <code>regulation_flags</code> 첨부
P3	문구-이미지 단절 제거	검증된 문구를 포스터 합성 단계로 직결, 좌표·폰트까지 한 흐름

2-4. 차별점과 설계 원칙

핵심 차별점은 **""예쁜 문구"**가 아니라 **"법적으로 문제없는 문구"**임. 세 원칙으로 구현함.

원칙	구현
막는 도구가 아니라 고쳐주는 도구	<code>block / warn</code> 2단계 판정 + 대체 표현 동시 제공
놓치는 비용이 잡는 비용보다 크다	하이브리드 신뢰 정책을 단조(monotonic)로 고정 — 최종 등급은 룰 1차 등급보다 낮아지지 않는다
커버리지는 열거 가능해야 함	규제 커버리지 매트릭스(카테고리 × 위반 축) 실측 → 갭을 우리가 먼저 선언

3. 서비스 소개

3-1. 기능 요구사항

구분	기능	설명	우선순위	구현
챗봇	업종 유형 분기	0단계에서 제품형/서비스형을 나눠 이후 흐름을 분리	필수	완료
챗봇	순차 슬롯필링	업종·용도·느낌·강조점·추가요청을 단계별 수집	필수	완료
챗봇	제품 Vision 인식	제품 사진에서 제품명 추출 후 사용자 확인	필수	완료
챗봇	조건부 후속질문	가격·확인처럼 사실값이 필요한 키워드에만 되묻기	선택	완료
문구	시안 3개 생성 · 직접 수정	headline 20자 / sub 30자, 다양성 임계 0.25, 수정 시 재검증	필수	완료
규제	룰 기반 자동 검사	카테고리별 정규식 룰 + 화이트리스트 오답 방어	필수	완료
규제	block/warn 2단계 판정	근거 법령 + 대체 표현 동시 반환	필수	완료
규제	LLM 2차 재검증	경계 케이스만 에스컬레이션, 단조 신뢰 정책	필수	완료
규제	자유입력 즉시 검사	후속질문·추가요청 제출 시점에 검사	필수	완료
포스터	초안 3장 → 고품질 렌더링	SD1.5 768px 시안 3장 중 선택 1장만 SDXL로 재생성	필수	완료
포스터	제품 보존	rembg 마스크 + inpaint, 최종 해상도 원본 재합성	필수	완료
포스터	비율·배경 선택	1:1 / 3:1 / 3:4 캔버스, AI · 단색 · 그라데이션 배경	필수	완료
포스터	문구만 재합성	diffusion 없이 CPU에서 문구 위치·크기 재조정	선택	완료
프론트	실시간 문구 편집	드래그 자유 배치, 폰트 5종, 실시간 CSS 미리보기	필수	완료
프론트	규제 배지 · 결과 다운로드	block 빨강 / warn 노랑 + 대체 표현, base64 → blob	필수	완료
공통	AI 생성물 고지	진입 고지 · 결과 캡션 · 이미지 워터마크 3지점	필수	완료
공통	이력 저장 · 공유 · 회원 인증	생성물 저장, 재조회, 계정별 데이터 분리	보류	범위 제외

보류한 두 기능 — 이력 저장과 회원 인증은 스토리지·DB 스키마·업로드/조회 로직을 새로 만드는 작업이라 프로젝트 기간 안에 구현하기 힘들 것이라 판단함. 대신 서버를 전부 stateless로 두고 프론트가 상태를 들고 다니는 구조를 택함(6-1절). 사이드바에서 이력이 차지했을 자리는 "오늘의 광고 규제 한 줄"이라는 정적 순환 콘텐츠로 채움.

3-2. 비기능 요구사항

구분	요구	달성 상태
응답 시간	초안 생성이 사용자가 기다릴 수 있는 범위	실서버 기준 drafts 약 19초 / refine 약 50초를 로딩 A·B 연출 기준으로 반영. 실제 응답이 더 빠르면 즉시 전환하고, 더 늦으면 마지막 단계에서 응답 완료까지 대기
동시 요청	시연 중 동시 요청에 실패하지 않을 것	text2img 3건 동시 → 200 3/3 (수정 전 1/3)
재현성	같은 입력에 같은 결과 (시연 방어)	시각 프롬프트 temperature 0.2, 후속질문은 서버 상수 매핑
안전성	검증되지 않은 문구가 포스터에 실리지 않을 것	문구 블록 출처를 사용자 입력 또는 /generate/copy 결과로 제한
법적 고지	AI 생성물임을 사용자가 인지	AI기본법 제31조 대응 3지점

3-3. 지원 범위

유형	업종	입력 이미지	생성 방식
product	푸드·뷰티·굿즈	제품 사진 필수	img2img (inpaint)
service	학원·체육관·도장	사진 없이 진행	text2img

초기에는 제품 사진이 있는 3개 업종만 지원함. 2주차에 "사용자에게 제품 사진이 있다"는 전제 자체가 확장성을 묶고 있다고 판단해 text2img를 도입하고 무형 서비스업 2종을 추가함.

추가 업종은 "규제 필터가 차별점이므로 광고 규제가 강한 업종일수록 필터의 가치가 부각된다"는 기준으로 고름. 성형외과는 의료 광고 사전심의가 별도로 필요해 제외했고, 청소·스터디카페는 규제가 약해 제외함.

비율 × 배경 지원 매트릭스

유형	1:1 (SNS)	3:1 (배너)	3:4 (상세페이지)
product · AI 배경	지원	지원	미지원 → 심플로 직행
product · 심플 배경	지원	지원	지원
service	AI만 (심플 미지원)	용도 선택지 미노출	용도 선택지 미노출

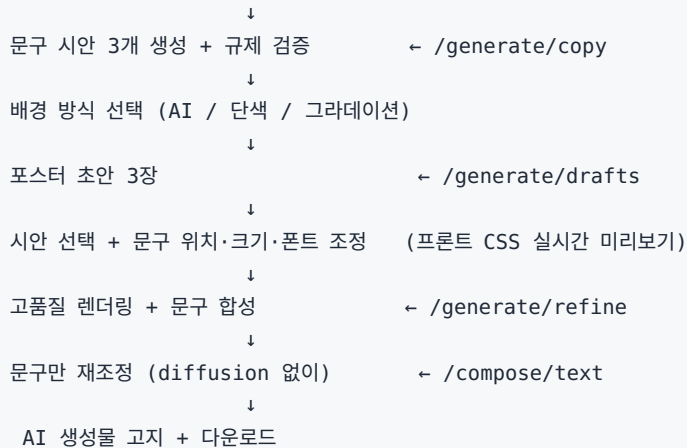
최종 출력 해상도 — 1:1 = 1024×1024, 3:1 = 3072×1024, 3:4 = 1024×1368.

3:4 AI 배경 미지원은 기술적 한계에서 나옴. 투명 제품(유리병)에서 AI가 제품 마스크 바깥으로 병목·뚜껑·바닥 구조를 이어 그리는 현상이 반복 확인됐고, 제품 위치를 옮겨도 다른 방향에 새 구조가 생겨 배치만으로는 해결되지 않았음. 불투명 제품 4종에서는 재현되지 않았으나 위험 제품을 자동 구분할 방법이 없어 열 수 없다고 판단함.

3-4. 사용자 시나리오

0단계. 업종 유형 선택 - 제품 광고 / 서비스 광고 (프론트 고정 질문)

- | | |
|--|--|
| <p>[product · 서버 6단계]</p> <p>1 업종 (푸드/뷰티/굿즈)</p> <p>2 용도 SNS 1:1 / 배너 3:1 / 상세 3:4</p> <p>3 제품 사진(필수) → Vision 인식 확인</p> <p>4 느낌 tone</p> <p>5 강조점 (+조건부 후속질문 1개)</p> <p>6 추가요청 (자유입력 40자)</p> | <p>[service · 서버 5단계]</p> <p>1 업종 (학원/체육관·도장)</p> <p>2 용도 SNS 1:1 고정</p> <p>3 느낌 tone</p> <p>4 강조점 (+조건부 후속질문 1개)</p> <p>5 추가요청 (자유입력 40자)</p> |
|--|--|



단계 수 표기 주의 — 서버 `FLOW_STEPS` 는 product 6 / service 5이고, 0단계는 프론트 고정 질문이라 `total_steps` 에 포함되지 않음. 문서에 "7/6"과 "6/5"가 섞여 나오는데 전자는 0단계 포함, 후자는 서버 기준임.

조건부 후속질문은 별도 단계가 아니라 강조점 단계의 서브질문이라 진행률 분모를 늘리지 않음. 가격·할인 같은 사실값 키워드를 고르면 서버가 결정적으로 우선순위 최상위 1개만 되물음.

3-5. 주요 화면

전체 흐름을 7개 화면으로 나눠 전부 구현함.

화면	역할	주요 구성
홈	진입 · 서비스 소개	사이드바(로고 / 새로 만들기 / FAQ / 오늘의 상식 / AI 생성물 고지), 예시 카드 티커, 마스코트
A · 챗봇	사용자 입력 수집	0단계 분기 후 product 6 / service 5단계 순차 질문, 진행률 dot 위젯, 사진 업로드, Vision 확인 UX
B · 문구 선택	문구 확정	시안 3개 중 선택, 규제 배지(block 빨강 / warn 노랑) + 대체 표현, 직접 수정 → 재검증
C · 배경·시안 선택	이미지 방향 확정	AI / 심플 배경 선택, 시안 3장 그리드, 선택 강조 + 확대 미리보기
D · 문구 편집	배치 조정	드래그 자유 배치(비율 좌표), 폰트 5종, 크기 프리셋, 안전 여백, 실시간 CSS 미리보기
E · 최종 결과	산출물 전달	AI 생성물 고지 캡션, base64 → blob 다운로드, 사이즈 토크
로딩 · FAQ	대기 · 안내	로딩 A(초안) / B(고품질화) 구분, 재시도·에러 UI / 요소별 스크린샷 포함 FAQ

UI/UX 스펙 문서(docs/UIUX_스펙정리.md)를 작성하고 지속 갱신해 **파트 간 계약의 단일 진실 원천**으로 삼음. 팀원 PR과 논의로 확정 사항이 바뀔 때마다 문서에 반영해, 프론트 구현과 실서버 연동이 같은 기준 위에서 진행되도록 함.

마감 구간에는 실서버 응답 시간을 기준으로 로딩 체크리스트 연출을 다시 맞춤. 초안 생성은 약 19초, 고품질화는 약 50초를 기준으로 하되 **연출이 API 완료를 지연시키지 않도록** 실제 응답이 먼저 끝나면 즉시 다음 화면으로 넘어가고, 응답이 늦으면 마지막 단계에서 완료까지 기다리도록 함. 홈과 배경 선택 카드의 placeholder도 실제 생성 예시로 교체해 최종 사용 흐름과 화면 설명이 같은 상태가 되도록 정리함.

3-6. AI 생성물 고지

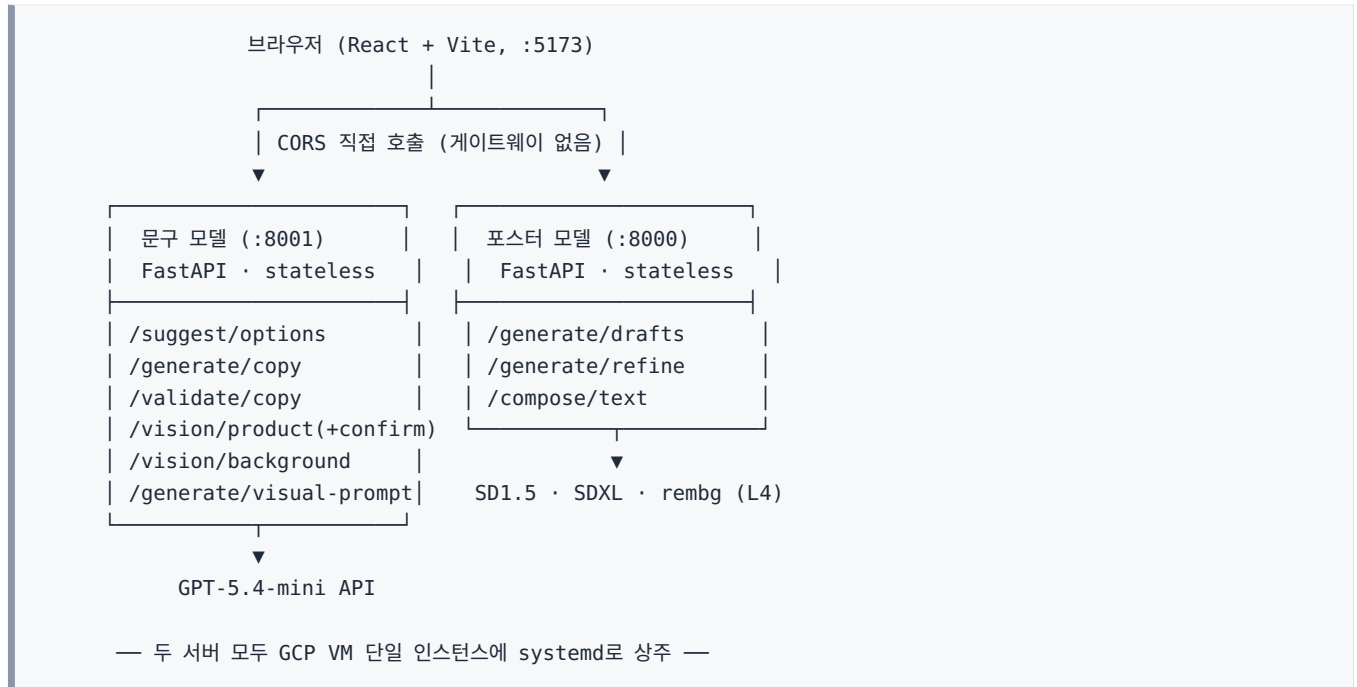
AI기본법 제31조에 따라 세 지점에서 대응함.

조항	지점	구현
①항	진입 화면	서비스 시작 전 사전 고지 배너
②항	결과 화면	결과 이미지 하단 캡션 텍스트
③항	산출물 자체	이미지 하단 반투명 워터마크 (다운로드본에 포함)

표준안은 docs/AI_생성물_고지_표준안.md 로 별도 문서화해 파트 간 구현 기준을 통일함.

4. 시스템 설계와 기술 선정

4-1. 전체 아키텍처



데이터 흐름을 글로 풀면 다음과 같음. 사용자가 챗봇에 답하면 프론트는 그 답을 `spec` 객체에 누적함. `spec` 이 채워지면 문구 모델의 `/generate/copy` 를 호출해 시안 3개와 규제 판정을 함께 받음. 사용자가 문구를 고르면 프론트는 같은 `spec` 으로 시각 프롬프트를 만들고(`/generate/visual-prompt`), 그 결과와 제품 이미지를 포스터 모델의 `/generate/drafts` 로 보내 초안 3장을 받음. 사용자가 한 장을 고르고 문구 위치를 조정하면, 프론트는 선택한 시안의 base64와 좌표·폰트를 `/generate/refine` 으로 보내 최종 이미지를 받음. 이후 문구만 다시 만지는 경우에는 diffusion을 타지 않는 `/compose/text` 로 감.

설계상 중요한 두 가지.

1. **서버는 모두 stateless임.** 프론트가 `spec` 과 선택한 시안 base64를 들고 다니며 재전송함. 이력 저장을 1차 범위에서 제외한 결정과 직결됨.
2. **게이트웨이가 없음.** 1차 서비스를 빠르게 완성하기 위한 선택이며, 인증·라우팅·요청 통제 지점이 없다는 트레이드오프를 감수함.

4-2. API 구조

문구 모델 — `/suggest/options` (챗봇 순차 질문, `input_type` · `max_length` · `regulation` 필드) / `/vision/product` (자동 확정하지 않고 `confirmation_required` 로 멈춤) / `/vision/product/confirm` / `/vision/background` / `/generate/copy` (시안 3개, headline 20자·sub 30자, 규제는 룰 단독) / `/validate/copy` (`use_llm=true` 면 룰+LLM 하이브리드) / `/generate/visual-prompt` (한국어 요청 → 영어 시각 프롬프트)

포스터 모델 — `/generate/drafts` (초안 3장, `aspect_ratio` · `placement` · `background_mode` · `subject_kind`) / `/generate/refine` (고품질화 + 문구 합성, `font_id` · `z_order` · 좌표 · `text.tone`) / `/compose/text`

필드	역할
<code>text.tone</code>	타이포 프리셋(<code>minimal_product</code> / <code>bold_promo</code>). 미지원 값은 diffusion 전에 400 <code>unknown_tone</code>
<code>subject_kind</code>	<code>product</code> / <code>service</code> . 서비스형에 제품 전용 접미사가 안 들어가게 하는 축. 기본값 <code>product</code> 라 기존 요청과 완전 호환

`text.tone` 은 챗봇 `spec.tone` (simple/warm/luxury/energetic)과 이름만 같고 다른 축임. 전자는 문구 타이포, 후자는 배경 프롬프트에 쓰임.

공통 규칙 — 이미지는 순수 base64(prefix 없음). 문구 좌표는 0~1 비율이며 기준점은 **텍스트 블록 중심**. `refine` 에는 `original_image` 를 항상 함께 보내 최종 해상도에서 원본 제품을 다시 합성함. 상태코드는 422 스키마 / 400 의미적 입력 오류 / 502 업스트림 / 500 설정 누락.

4-3. 이미지 생성 2단계 파이프라인

[1단계 draft] SD1.5 · 768px · 3장 → 사용자가 방향 선택
 [2단계 refine] SDXL · 선택한 1장만 → 문구 합성 + 워터마크
 [반복 편집] /compose/text · CPU 수백 ms → diffusion 없이 문구만 재합성

무거운 연산을 한 번만 쓰는 지점을 명확히 나눈 구조임. 근거와 검증 결과는 5-2절에 있음.

4-4. 규제 필터링 4중 방어

서비스의 핵심 설계임. 한 겹으로는 막을 수 없다고 보고 성격이 다른 네 겹을 쌓음.

- ① 생성 단계 차단 프롬프트에 금지 규칙 명시
↓
- ② 룰 기반 자동 검사 `regulation_flags` 자동 첨부 (비용 0, 즉시)
↓ + `whitelist` 오탐 방어 + `block/warn` 2단계 + 대체 표현
↓
- ③ LLM 2차 재검증 경계 케이스만 에스컬레이션 → 맥락 판단
↓ 단조(monotonic): 상향만 반영, `block` 완화 불가
↓
- ④ 챗봇 자유입력 즉시 검사 후속질문·추가요청을 제출 즉시 `check_rules`
 `block` → 재입력 / `warn` → 경고 동봉 후 진행

겹	성격	잡는 것	못 잡는 것
① 생성 억제	사전	모델이 위반 표현을 애초에 덜 만들게 함	사용자가 직접 요청하면 뚫림
② 룰 검사	결정적 · 비용 0	명시적 금지 표현, 정형 패턴	맥락 의존 표현, 의미 회피
③ LLM 재검증	맥락 판단	정규식이 놓친 의미 회피, 최상급 맥락	비용 발생 → 경계 케이스만
④ 자유입력 검사	입구 통제	사용자가 직접 넣는 위반 문구	—

④는 3주차에 추가됨. Design Planner가 규제 검증을 거치지 않은 문구를 포스터에 실을 수 있다는 구조적 위험(8-4절)을 발견하면서 "포스터에 실리는 모든 문구는 사용자 입력 또는 `/generate/copy` 결과에만 근거한다"는 원칙이 세워졌고, 그 사용자 입력 경로 자체에도 검사가 필요해졌기 때문임.

4-5. 기술 스택

기술을 나열하는 대신 **왜 골랐는지**를 함께 적음.

분야	기술	선정 이유
Frontend	React 19 + Vite	상태가 많은 다단계 편집 UI. Vite는 개발 서버 기동이 빨라 실연동 디버깅 반복에 유리
Backend	FastAPI	모델 서빙과 API를 한 프로세스에서 다룸. 전후처리(rembg·합성·워터마크)가 복잡해 자유도가 필요
Language	Python 3.11	diffusers·transformers·PIL 생태계
문구 LLM	GPT-5.4-mini (API)	4-6절 참조
이미지 모델	SD1.5 + SDXL	4-6절 참조
배경 제거	rembg	제품 마스크 추출. 마스크 품질이 inpaint 결과를 좌우
폰트	Pretendard 외 5종	fontTools로 18종 검증 후 한글 전수 지원 기준으로 축소
배포	GCP VM (g2-standard-4 + NVIDIA L4)	제공 크레딧 내 무료. Cloud Run은 11GB 적재로 콜드 스타트 손해가 큼
프로세스 관리	systemd	tmux는 SSH 끊김만 막고 프로세스 종료는 복구 못 함 (실제로 한 번 다운)
협업	GitHub Pull Request · Discussions	코드 리뷰 승인 후 병합, 게시판 템플릿으로 협업 일지 운영

DB가 없는 이유 — 이력 저장을 1차 범위에서 제외해 영속 데이터가 없음. 서버는 전부 stateless이고 상태는 프론트가 들고 다님. 이력·공유·검색을 열 때 DB 도입이 함께 필요해진다(9장).

4-6. AI 모델 선정과 이유

역할	채택	선정 이유	검토한 대안
문구 생성·규제 2차 판정	GPT-5.4-mini	한국어 광고 카피 품질이 충분하고, 20자/30자 같은 길이 제약과 JSON 출력 지시를 안정적으로 따름. 규제 판정은 근거 문장을 함께 요구하므로 지시 수행력이 중요	자체 LoRA 파인튜닝 — 데이터 수집·학습 시간 대비 개선폭이 작다고 판단해 프롬프트 엔지니어링으로 전환
초안 이미지 (draft)	SD1.5	768px 3장을 뽑는 데 실측 18.3초. 방향 선택에 필요한 정보(구도·색감·분위기)는 이 해상도로 충분히 전달됨	SDXL로 draft — 3장 생성 시간이 몇 배가 되어 사용자 대기가 과도
고품질 렌더링 (refine)	SDXL	최종 출력 해상도(1024~3072px)에서 디테일 확보	업스케일만 — 대조군 실험 결과 오히려 선명도가 나은 경우가 있어 수단으로 확정하지 않은 상태 (5-2절)
제품 마스크 추출	rembg (U²-Net)	별도 학습 없이 제품 누끼가 실용 수준. inpaint 마스크로 바로 사용 가능	ControlNet(canny) — 형태만 전달되고 색·재질이 보존되지 않아 폐기

모델을 학습시키지 않은 이유. 이 프로젝트에서 학습(fine-tuning)을 수행한 모델은 없음. 문구는 프롬프트 엔지니어링, 이미지는 사전학습 모델 + 후처리 조합임. 3주 일정에서 데이터 수집·학습·평가 루프를 도는 것보다, **규제 검증 데이터셋(골드셋)을 제대로 만들고 평가 체계를 정직하게 세우는 쪽이 서비스 가치에 직결된다고 판단함.** 그 결과 이 프로젝트의 ML 작업은 학습이 아니라 **평가 설계**에 집중돼 있음(5-1절, 7-1절).

4-7. 데이터

평가 데이터(골드셋)와 규칙 데이터(규제 룰 사전) 두 종류가 있음.

데이터	규모	출처 · 구축 방법	용도
규제 룰 사전	고유 50개 (COMMON 9 + 카테 고리 전용 41)	법령 원문과 공정위 제재 사례를 읽고 수작업 작성. 룰마다 패턴 (정 규식) · severity · 근거 법령 · 대체 표현 4요소	룰 기반 1차 검사
골드셋	127건 (train 71 / holdout 43 / redteam 13)	위반·경계·정상 문구를 직접 작성하고 3등급 라벨링	규제 필터 성능 측정
화이트리스트	카테고리별 허용 표 현	오탐 사례에서 역으로 수집	정규식 오탐 방어
폰트 메타	18종 ttf	fontTools로 글리프 커버리지 실측	렌더링 깨짐 사전 차단

분할 원칙 — train은 룰을 고치면서 함께 본 셋이라 100%가 나오는 것이 당연하고 성능이 아님. **holdout 43건이 실제 성능 대표 값**이며, 룰을 수정할 때 열어보지 않았음. redteam 13건은 의미 회피 스트레스셋으로 대표 지표에 넣지 않음. 마지막으로 골드셋을 확충할 때도 신규 9건을 전부 train에만 넣고 holdout 43건은 원본 그대로 보존함.

5. 기술 구현

5-1. 문구 모델 · 규제 검증 · 챗봇 (김도혁)

저장소 `copy_model/` . 주차별 측은 1주차 만들기 → 2주차 신뢰도 세우기 → 3주차 커버리지 증명하기로 이동함.

규제 룰 사전

각 카테고리 = 공통(COMMON, 표시광고법 9개) + 카테고리 전용 룰.

category	전용	합계	block	warn	주요 근거 법령
food	11	20	6	14	표시광고법, 식품표시광고법, 건강기능식품법
beauty	9	18	5	13	표시광고법, 화장품법
goods	11	20	3	17	표시광고법, 상표법, 약사법, 의료기기법
sports	6	15	2	13	방문판매법, 국민체육진흥법, 의료법
academy	4	13	2	11	표시광고법, 공정위 제재 사례, 학원법 시행령

고유 룰 총 50개(COMMON 9 + 전용 41).

규제 커버리지 매트릭스 — "커버리지를 열거 가능하게"

"룰 개수 ÷ 케이스 수로 커버리지를 판단하면 룰이 동작하는 건 알라도 어디까지 동작하는지는 모른다"

담당자가 "케이스 수 ÷ 룰 수만으로 보면 왜곡될 수 있다"고 답하며 대안으로 **rule coverage matrix(카테고리 × 위반 측)** 를 제안했고 채택됨. 매트릭스를 실제로 그리자 갭이 드러남.

갭	내용	처리
G1	goods block 룰 0개 — 전 카테고리 중 유일	해소 (0→3)
G2	의료행위 오인 측이 어느 카테고리에도 없음	해소
G3	academy 계약 불공정(환불) 측 공백	해소
G4~G7	academy 근거법 단일 / whitelist 편중 / 다국어 미지원 / 실증 책임 미구조화	향후 과제

G1에는 반전이 있었음. 최초 가설은 "굿즈의 첫 block은 위조품·상표 측"이었으나 실제로는 **의약품·의료기기 오인 측**이 더 명확함. 게르마늄 팔찌·자세교정 방식처럼 일반 상품이 질병 치료를 표방하는 경우는 위법성이 분명한 반면, 레플리카 광고 자체의 위법 판단은 사안별이라 향후 과제로 남김.

평가 체계와 성능

골드셋 127건(train 71 / holdout 43 / redteam 13)으로 측정함. 대표값은 ****HOLDOUT 라벨 정확일치 93.0%****이며 TRAIN 100%는 룰을 고치면서 함께 본 셋이라 성능이 아님. REDTEAM 23.1%는 의미 회피 스트레스셋으로 대표 지표에 포함하지 않는다 (전체 지표는 7-1절).

남은 오류(정직한 한계) — 미탐 1건("면역 쑥쑥 올려주는 홍삼": 의미 회피), 오탐 2건("최고의 하루를 담은 다이어리", "완벽한 오후를 위한 캔들": 최상급이 제품이 아니라 하루·오후를 수식하는데 정구식이 맥락을 못 봄). 셋 다 원인이 같아 LLM 2차 레이어 필요성의 실측 근거가 됨.

A/B 실험 — 규제 지침의 효과

그룹	위반율	경고율
A · 규칙 없음	2.8%	19.4%
B · 규칙 있음	0.0%	8.3%

v1 실험은 A·B 모두 위반율 0%로 실패함. 원인은 "규칙이 무효"가 아니라 **무자극 입력이라 모델이 위반할 이유가 없었던 실험 설계 결함**이었음. "소상공인이 위반 표현을 직접 요청하는 상황"으로 재설계한 뒤에야 효과가 측정됨.

LLM 2차 검증 레이어 (하이브리드)

정구식은 맥락을 못 보고, 전건을 LLM에 보내면 비용이 폭발함. **경계 케이스만 에스컬레이션**하는 구조를 설계함(block은 룰 신뢰 / warn+최상급 어휘 → LLM / safe+효능 회피 어휘 → LLM / 그 외 생략).

라우팅 정책 2종 중 정확도를 우선해 **broad를 운영 기본값**으로 채택했고 `lexicon` 은 "비용 절감 고도화 전략"으로 남김. 실측(24콜)에서 HOLDOUT 90.6% → **100%**, REDTEAM 27.3% → **63.6%**, **2차 오판 0건**이었음.

신뢰 정책 — 비대칭에서 단조로

가장 중요한 설계 변경임. 원래는 "LLM이 근거를 대면 warn→safe 하향 허용"이었는데 리뷰에서 두 방향의 지적이 겹침. 하향 가드가 `verdict.reason.strip()` 으로 **근거 문자열의 존재 여부만 보는 형식 검사**인데, 판정 프롬프트가 reason을 필수로 요구해 LLM이 근거를 거의 항상 채워 반환하고 있었음. 결과적으로 어휘 조건을 통과한 warn은 LLM 판정이 사실상 최종 등급이었음.

내용 검증을 새로 만드는 대신 **하향을 전면 차단하는 단순·안전 정책**을 택함.

```
final_severity = max(rule_severity, llm_severity)
→ LLM은 상향 또는 동일만 반영. block은 어떤 근거로도 완화되지 않음.
```

"규제 도구는 놓쳤을 때의 비용이 크다"를 코드로 고정한 것임. 검증도 `_client_chat` 을 patch해 실제 LLM 호출 경로까지 훑어내는 통합 테스트로 강화함(신규 11건, 관련 27건 통과).

챗봇 로직

- **business_type 분기(2주차)** — 0단계에서 제품형/서비스형을 나누고, 각 대화 단계가 자기 슬롯만 고치도록 막아 뒷 단계가 앞 값을 덮어쓰는 일을 방지
- **제품 Vision 인식** — 원칙은 **"AI는 보이는 것만 말하고 최종 결정은 서버가 한다"**. 효능·성분처럼 사진에 없는 정보를 지어내지 못하게 차단. 초기 설계는 자동 확정이었으나 "잘못 인식하면 강조점·문구·포스터까지 연쇄 오류"라는 리뷰 지적으로 확인 UX와 전용 `confirm` 엔드포인트를 추가
- **강조점 후속질문(3주차)** — 가격·할인율은 사용자가 알려주지 않으면 알 방법이 없고, 그러면 **없는 숫자를 지어내는 위험**이 생김. 질문 선택은 LLM이 아니라 **서버 상수 매핑**이라 같은 키워드면 항상 같은 질문이 나와 시연 재현성이 확보됨. 수집값은 `source="user_input"` 으로 태깅되어 Claim-Lock의 입력 경로가 됨

Claim-Locked Copy 설계(미구현) — LLM이 근거 없는 사실을 지어내 규제를 우회하는 것을 막는 설계. 핵심 불변식은 **"문구가 주장하는 모든 사실은 출처가 LLM이 아니어야 함"**이며, PM 리뷰를 거쳐 수집값마다 출처를 기록하는 필드가 추가되어 이를 코드로 검증할 근거가 생김.

매력 문구 A/B 실측

3주차에 PM이 요청한 "규제 정확도 → 카피 매력도" 전환의 실행임. 실험 코드가 저장소에 없던 이유부터 규명함. 앞서 별도 실험 브랜치에서 다루기로 하고 제외했는데 **그 브랜치가 만들어지지 않아 문서만 남고 코드가 빠진 상태**였음. 커밋 이력에서 복원해 현재 인터페이스에 맞게 정리함.

구조는 두 축을 함께 측정함 — 변형별 **규제 통과율을 룰 검사로 결정적으로 재고**(비용 0), 변형 vs baseline을 LLM 심사위원이 **A/B 자리를 바꿔 pairwise** 판정해 승률을 산출함. **baseline보다 위반율을 올리지 않으면서 승률 0.5를 넘는 변형만** 채택 후보가 됨.

변형	승률	판정	위반율(block)	safe	글자수 준수
baseline	—	—	0	100%	1.0
persona	0.708	17/24	0	100%	1.0
hook	0.667	16/24	0	100%	1.0
cta	0.583	14/24	0	100%	0.861

셋 다 위반율을 올리지 않아 형식상 전부 채택 후보지만, **persona가 0.708로 가장 우세했고 재측정에서도 일관되게 우세함**. cta만 글자수 준수율이 떨어짐. 프로덕션 프롬프트 반영 여부는 팀 논의 과제로 남음.

영어 시각 프롬프트 계층 (황지우)

학원·체육관을 요청해도 업종과 무관한 비슷한 실내가 생성되던 문제의 근본 원인이었음. **이미지 모델에 전달되는 프롬프트가 전부 한국어**였고, SD1.5의 텍스트 인코더는 영어 기준이라 한국어 업종 정보가 거의 반영되지 않았음. 업종이 달라도 모델 입장에서는 같은 지시를 받는 셈이었음.

KO "복싱 체육관 회원 모집" → 거실에 가까운 실내
 EN "boxing gym interior with punching bags" → 실제 체육관

모델의 표현력 문제가 아니라 입력이 전달되지 않는 문제였음. 한국어 요청을 배경·조명·색감·분위기·구성 축으로 정리해 짧은 영어로 조립하는 `/generate/visual-prompt` 를 신설함. 기존 로직은 수정하지 않고 라우트만 추가했으며, 기존 클라이언트를 재사용해 앞서 도입한 타임아웃·재시도 정책을 그대로 상속함. temperature는 문구 생성(0.9)과 달리 **0.2**로 둬 — 같은 입력에 같은 결과가 나와야 시연이 재현되고 시안 3장의 차이를 seed 탕으로 볼 수 있기 때문임.

관례에서 벗어난 부분도 문서에 남김. **이 엔드포인트만 키가 없어도 500을 내지 않음**. 다른 라우트는 키가 없으면 문구를 만들 수 없어 500이 맞지만, 이쪽은 업종별 영어 기본값으로 동작하므로 500을 내면 그 fallback 경로에 영영 도달하지 못함.

업종명 제거 규칙

학원 배경이 강의실보다 로비·라운지로 나오는 문제가 남음. `subject_treatment` 에 `academy interior space` 가 들어가는데 **이미지 모델이 `academy` 를 강의실로 해석하지 못함**. 반면 `gym` 은 단어 자체가 운동기구 있는 공간을 가리켜 정상 생성됨.

단계	subject_treatment	생성 경향
현재	<code>academy interior space</code>	로비 · 거실에 가까운 이미지
1차 시도	<code>academy interior with desks and whiteboard</code>	사무실 · 코워킹
적용	<code>classroom interior with desks and whiteboard</code>	강의실

구체적인 사물을 추가하는 것만으로는 부족했고, **업종명 자체를 제거해야 한다**는 점이 핵심임.

동시 요청 지연 대응과 문서 수치 정정은 8-3절에 함께 정리함.

5-2. 포스터 생성 모델 (항지우)

저장소 `poster_model/` . 테스트 파일 26개.

2단계 파이프라인 — SD1.5 draft → SDXL refine

SD1.5로 768px 시안 3장을 뽑고, 사용자가 고른 한 장만 SDXL로 고품질 렌더링하는 구조임. 프로젝트 초기에 정해져 3주 내내 유지됨.

아래는 당시의 후보 비교 기록이 아니라, 3주간 구현·운영·검증을 거친 뒤 이 구조가 유효했던 지점을 정리한 해석임. 초기에 어떤 대안들을 놓고 비교했는지는 별도로 남아 있지 않음.

- ① **실측 소요가 두 단계에서 크게 다름**. 같은 조건(text2img, beauty)에서 draft 18.28초, refine 53.22초. 시안 단계에 무거운 모델을 쓰면 3장 뽑는 시간이 그대로 몇 배가 됨. 반면 방향을 고르는 데 필요한 정보(구도·색감·분위기)는 768px로 전달됨.
- ② **대기가 두 덩어리로 나뉘고 그 사이에 사용자가 개입함**. 프론트 로딩 화면을 **A(초안)** / **B(고품질화)** 로 나눈 것이 이 구조의 반영임. 총 소요가 같아도 중간에 고르는 행동이 들어가면 아무것도 못 하고 기다리는 구간이 짧아짐.
- ③ **반복 편집이 diffusion을 다시 타지 않음**. 드래그 중에는 CSS로만 미리보기하고 [완성하기] 시점에 refine을 1회 호출함. 확정 후 문구만 조정하는 경로는 `/compose/text` 로 분리해 **CPU 수백 ms**에 처리함.
- ④ **구조와 수단이 분리돼 있음**. 3주차 검증에서 refine 수단(SDXL img2img)은 재검토 대상이 됐지만 draft → 선택 → refine 흐름 자체는 유지함. 수단을 바꿔도 구조가 성립한다는 것이 확인된 셈임.

트레이드오프는 두 모델을 함께 다뤄야 한다는 점임. VRAM 부담이 늘고(피크 7,864 MiB), refine이 사용자가 고른 시안의 구조를 재해석하는 문제도 여기서 생김.

제품 보존 — ControlNet에서 inpaint로

초기 ControlNet(canny)는 **형태만 전달되고 색·재질이 보존되지 않았음**(레드벨벳 케이크가 분홍 무스로 변형). rembg 마스크 + inpainting으로 전환해 해결했고, 이후 핵심 원칙이 된 것은 **"제품은 항상 최종 해상도의 원본으로 다시 합성한다"** 임. 배경 생성 해상도와 제품 선명도를 분리하는 이 원칙 덕분에, 생성 해상도를 낮춰 속도·메모리를 확보해도 로고·패키지 문구 선명도는 유지됨.

비정사각 지원 — canvas bridge와 footprint

기존 파이프라인은 내부 여러 곳에서 정사각을 강제해 1:1 외 용도를 만들 수 없었고, 목표 비율로 바로 잘라내면 3:1에서 세로형 제품이 잘림. **제품 추출용 정사각 소스 처리와 최종 출력 캔버스를 분리하고 그 사이를 잇는 canvas bridge로 해결함.**

제품 배치는 bbox가 아니라 **그림자까지 포함한 footprint**로 계산함. 접지 그림자가 제품 폭에 비례해 아래로 뻗어 bbox만 보면 잘리기 때문임. **벗어나는 값은 보정하지 않고 거부함(fail-closed)**. refine에서는 draft 크기로 비율을 추론하고 **명시값과 추론값이 다르면 차단함** — 3:1 시안을 골랐는데 필드 누락으로 1:1이 나오는 조용한 실패를 막기 위해서임.

서비스형 레이아웃 — 생성에서 후처리로

서비스형은 공간 자체가 피사체라 화면이 가득 차 문구 자리가 없음. 프롬프트로 "여백을 남겨라"를 지시하는 방식은 **표현 변경·프레이밍 좌표 지시·카메라 축 변경까지 세 라운드를 돌렸으나 전부 실패함**. 여백이 생겨도 위치가 매번 달랐음.

원인 진단이 정확함. **생성 모델에 결정론적 배치를 요구한 것 자체가 잘못된 접근이었음**. CLIP은 부정 표현을 읽지 못하고 3D 공간 상대 위치도 해석하지 못함. 그래서 **생성 후 후처리로 슬롯을 만드는 방식으로 전환함**.

비율	캔버스	밴드 / 패널	배경 crop
1:1	768×768	상단 184px (짧은 변의 0.24)	상단 184px
3:4	768×1024	상단 256px	없음
3:1	2304×768	좌측 패널 1536px (67%)	없음

3:1의 좌측 패널 67%는 설계 단계에서 "단색 배너 옆에 사진"이 된다고 우려해 제외 후보였음. 그런데 **비중 수치만으로 자르지 않고 6장을 실제로 합성해 확인**했더니 결함이 아니었음. 3:1 자체가 배너 비율이라 좌 카피 / 우 이미지가 관습적으로 읽히고 이미지 33%로도 두 업종이 식별됨. 부수적으로 **그림체가 크게 다른 24장 전부에 확정 레이아웃을 적용해 문구 축소 0건**을 확인해, 레이아웃이 그림체와 독립임이 실측됨.

고품질화 재검토 — 대조군이 판단을 뒤집다

2단계 파이프라인이 실제로 화질을 올리는지 검증한 결과 **구조는 상당히 재해석하면서 화질 이득은 뚜렷하지 않았음**. 검증 경위는 8-2절에 있음.

원인은 파라미터가 아니라 구조였음. img2img가 구조를 지키는 수단은 denoise를 덜 하는 것뿐이라 구조 유지와 재생성 폭이 하나의 값으로 동시에 움직임. 적게 바꾸면 좋아질 여지가 없고, 많이 바꾸면 사용자가 고른 그림이 아니게 됨.

SD1.5가 상한인 것도, SDXL로 교체하면 되는 것도 아님. **사용자가 고른 시안의 공간 구조와 업종 의미를 유지한 채 SDXL 품질을 얻는 방법이 없다는 것이 문제임**. 화질 문제가 아니라 **조건부 재생성 문제임**.

다음 방향으로 **ControlNet Depth**를 1순위로 정함. canny가 아닌 이유는 현재 draft의 세부 윤곽 자체가 왜곡된 경우가 있어 canny로 주면 **SDXL이 왜곡까지 고착**시키기 때문임.

Design Planner Core

포스터 디자인 결정을 데이터로 구조화하는 실험적 계층임.

```
CreativeBrief → Planner → RenderSpec → validate → RenderPlan → Renderer
                                     → RenderEvidence → Safety
```

지켜지는 계약 중 두 가지가 중요함 — **Safety**는 관측만 함(자동 retry·보정을 설계상 넣지 않는다), **production**과 분리됨 (dynamic/ 은 api.py 에서 호출 0건).

Planner가 정상적으로 값을 출력하는 것과 실제 광고 포스터의 품질이 좋은 것은 별개의 문제임. 기술적 오류를 모두 해결 해도 최종 포스터의 구성과 완성도가 충분하지 않은 사례가 확인됨. 기능 부족이 아니라 **Planner가 처음부터 어떤 디자인을 만드느냐**가 더 중요한 개선 지점임.

그래서 우선 보완 대상을 초기 **Planner**의 구성·정보 위계·제품과 문구의 관계 설계로 정리하고 후처리 보정은 뒤로 미룸.

동시성과 프롬프트 예산

두 건 모두 실서버에서만 드러난 문제이며, 증상·원인·결과는 8-3절에 있음. 여기서는 **고치는 방식에서 내린 판단**을 정리함.

pipe_lock.py — pipe 사용 구간 **전체**를 `threading.RLock` 으로 직렬화되 함수 전체는 잠그지 않았음. solid/gradient 초기 반환 경로는 pipe를 만지지 않는데, 함수 전체를 잠그면 **고칠 이유가 없는 요청까지 앞선 diffusion 30초를 기다림**. 잠글 대상을 예외가 나는 tokenizer로 좁히지 않은 이유는 아래와 같음.

예외가 사라지는 것과 안전해지는 것은 다름. scheduler 등 pipe 가변 상태 공유는 예외를 내지 않음 — 조용히 잘못된 이미 지를 만듦. **눈에 안 보이는 실패가 눈에 보이는 실패보다 낫지 않음.**

배포 전제(single worker)는 코드 상수 `SINGLE_WORKER_ONLY = True` 로 명시해, worker를 늘리면 이 Lock이 무력하다는 점을 남김.

prompt_budget.py — 최종 프롬프트는 quality baseline + 사용자 prompt + 경로별 constraint 임. 문구 모델은 자기 조 각만 알아 전체 길이를 모르므로 **전체를 아는 포스터 모델이 예산 보장의 책임을 짐.**

- **보호 영역**(quality baseline, 제품 constraint) — 절대 버리지 않음. 실서버에서 잘려 나갔던 것이 정확히 이 부분임
- **가변 영역**(사용자 prompt 조각) — 예산이 모자라면 ", " 단위로 **뒤에서부터** 버림
- target 72 / hard limit 77 — 경계에 붙여 두면 사소한 문구 수정이 곧바로 절단이 되므로 77을 딱 채우지 않음
- 문자 수 추정치 아니라 **실제 tokenizer로 센다**(SDXL은 tokenizer 2개 중 큰 값)

하지 않기로 한 것도 남김 — `model_max_length` 를 임의로 늘리지 않음(숫자만 바꾸면 으페), Compel·chunking을 도입하지 않 음, 잘린 사실을 조용히 넘기지 않음.

budgeter가 뒤에서부터 버리므로 기존 순서에서는 **사용자가 직접 입력한 request 가 가장 먼저 버려짐**. 순서를 바꿔 request를 마지막까지 보존하도록 했고, 무해한 정렬 변경이 아니라 **의도적인 prompt-priority 변경**임을 주석에 명시함.

```
기존: [tone, product, keywords, request]
변경: [request, product, keywords, tone] ← request가 마지막까지 보존
```

이 작업이 남긴 재검증 과제 — 기존 디자인 실험에서 "모델이 특정 지시를 잘 따르지 않는다"고 판단했던 결과 중 일부는, **모델 성능이 아니라 prompt가 끝까지 전달되지 않았기 때문일 가능성이** 생김. 그림체 지원 범위와 고품질화 결론 일부가 truncation된 조건에서 측정된 것일 수 있음. 담당자는 이를 단순 서버 오류 수정으로 끝내지 않고 팀에 공유해 디자인 실험과 품질 재검증의 순서를 다시 정리함.

실서버 안정화 — 프리셋 · 메모리 · 서비스형 파라미터

항목	문제	조치 · 판단	결과
타이포 프리셋	TONE_PRESETS 와 <code>render_text()</code> 에 기능이 있는데 API 경로만 그것을 쓰지 않아 두 톤이 같은 스타일이 됨. 검증 스크립트는 프리셋을 직접 읽어 실험 결과와 실제 API 결과가 달라질 수 있는 상태	규칙 셋을 고정 — 개별 필드가 프리셋을 이긴다(프론트 미리보기와 어긋나면 안 됨), 폴백하지 않음(미지원 tone은 400), 검증은 diffusion 전에(렌더링 시점에 터지면 GPU 시간을 통째로 버림)	두 톤이 실제로 구분됨
메모리 부족	고품질화를 반복하면 포스터 서버 프로세스가 강제 종료됨 (7회 중 2회)	① VRAM 상주로 전환(RAM 10.2GB→2GB) ② warmup에서 SDXL 제외(상주 20.4→6.4GB) ③ SDXL 변형 1개만 상주 — 제품형 inpaint와 서비스형 img2img는 동시에 필요하지 않음. <code>to("cpu")</code> 즉시 반환안은 VRAM 동일한데 전환만 악화(48→70초)돼 기각	6/6 통과, 전환 48.7 → 9.9초
서비스형 배경	서비스형 배경 원근 왜곡 — refine을 거쳐도 해상도만 오르고 어색함이 남음	제품형 기준 두 값이 서비스형에 그대로 적용되던 것이 원인. 두 값 모두 "보존할 원본 제품이 있다"는 전제인데 서비스형은 text2img라 앵커가 없음 → 시안 해상도 768→512, refine strength 0.35→0.75	시간 3.4→1.7초. 제품형 회귀는 동일 시드 MD5 일치로 확인

VRAM 21,402 MiB는 누수가 아님 — 모델 상주 위치를 호스트 RAM에서 VRAM으로 옮긴 결과이며, PyTorch 캐싱 할당자의 정상 상태(가중치 13.4GB + 재사용 워크스페이스 약 8GB)임. 이 장비는 VRAM 24GB / RAM 16GB라 CPU offload가 오히려 불리함.

인물 배제 — 그리고 근거 재검증

서비스형 배경에 인물이 등장함. 제품형은 제품만 그리도록 지시하는 접미사가 부수적으로 막고 있었지만 서비스형은 막는 장치가 없었음. 공용 negative 프롬프트에 인물 토큰을 추가했는데, **적용 후 확인 과정에서 당시 근거가 현재 조건에서는 성립하지 않는다는 것이 드러남**. 초기 검증에 쓴 `academy interior space` 는 앞서 업종명 제거 규칙을 적용하면서 이미 바뀐 문자열이었음.

프롬프트	인물 출현
<code>academy interior space</code> (이전 문구)	12개 중 2개
<code>classroom interior with desks and whiteboard</code> (현재 문구)	12개 중 0개
<code>gym interior with exercise equipment</code> (현재 문구)	12개 중 0개

원인은 **negative 프롬프트가 아니라 장면을 결정하는 positive 프롬프트 쪽이었고**, 업종명 제거 규칙에서 이미 해소된 상태였음. 제품형에서는 negative 변경만으로 배경이 평균 8.45/255, 변화 픽셀 58.7%만큼 달라졌는데 **개선인지 악화인지는 표본이 시드 1개뿐이라 판단하지 않았음**. 효과는 확인되지 않고 결과만 달라지는 상태여서 제품형에서는 제외하고 서비스형 전용으로 분리함.

서비스형에서도 현재는 막을 대상이 없지만 유지한 이유를 남김 — **24회 모두 나오지 않았다는 것이 향후 미출현을 보장하지 않고**, 문구 모델이 항상 현재 형태의 프롬프트를 산출한다는 전제에만 의존하게 되며, 사용자 직접 입력도 같은 경로로 들어오기 때문임. 측정 범위의 한계(draft 단계만, 프롬프트 2종, 육안 판정)도 함께 기록함.

subject_kind 프론트 배선

서비스형 요청이 제품 사진처럼 생성되던 문제임. 파라미터는 서버에 이미 추가돼 있었고 분기와 테스트도 갖춰져 있었으나, **프론트가 값을 보내지 않아 서버가 기본값 product 로 처리**하고 있었음. positive 접미사

single product only, isolated product shot, no other objects 가 사물 하나만 그리도록 강제해 회색 배경에 물건 하나가 나옴. **서버는 건드리지 않고** 프론트가 spec.business_type 을 서버 계약인 subject_kind 로 옮기도록 배선만 추가함. 서버에 기능이 있어도 **호출부에 배선이 없으면 한 번도 적용되지 않는다는 것**을 보여준 사례임.

5-3. 프론트엔드 · 백엔드 연동 (김소원)

화면 구성, API 연동, 오케스트레이션, 결과 표시·편집·다운로드를 담당함. React 19 + Vite 기반임.

챗봇을 메인 인터페이스로

카드/칩 선택형을 메인에 두고 챗봇을 우측 보조 패널로 쓰는 안과, 챗봇 자체를 메인으로 쓰는 안을 놓고 **두 버전 목업을 만들어 비교**함. 문구 모델의 /suggest/options 가 고정 순차 질문 구조라 특정 항목만 묻는 보조 도우미로는 쓸 수 없었고, 목업 비교에서도 메인형이 선택 과정이 간결하고 카테고리 밖 선택지 등록이 자연스러웠음.

트레이드오프는 카드형 대비 **여러 선택지를 한눈에 훑는 경험이 줄어든다**는 것임. 진행률 표시와 단계 간 자동 연결로 순차 진행의 답답함을 보완함.

3주차에는 신규 챗봇 API 계약(input_type · max_length · regulation)을 연동해 **조건부 후속질문 · 자유입력 · 규제 안내 UI**를 계약 수준에서 구분해 처리하도록 재구성함. 후속질문은 별도 step으로 늘리지 않고 강조점 단계의 서브질문으로 처리해 **전체 진행 단계가 불필요하게 길어지지 않게** 함.

product / service 분기와 Vision 흐름

기존에는 제품과 서비스를 하나의 흐름에서 다뤄 네일샵처럼 경계가 애매한 업종에서 선택지가 모호해짐. 첫 질문에 business_type 을 추가해 흐름 자체를 나눔. LLM이 업종을 유동적으로 분류하게 두는 안도 있었으나, **이후 이미지 생성 방식이 product와 service에서 다르기 때문에 초기에 경계를 명확히 하는 편이 안정적**이라고 판단함.

product에서는 제품명 텍스트 질문을 제거하고 **제품 사진을 Vision이 인식**하게 바꿈. 어차피 포스터 생성에 제품 사진이 필요하므로 같은 이미지를 재활용하면 사용자 입력을 하나 줄이면서 정보 정확도를 높일 수 있음.

다만 인식 결과를 그대로 확정하지 않았음. 제품명이 잘못 인식되면 **강조점 → 문구 → 최종 포스터까지 연쇄적으로 잘못된 정보 위에서 진행**되기 때문임. [맞아요] / [수정할게요] / 재업로드로 보정하는 확인 UX를 넣었고, 이 요구가 백엔드의 전용 확정 엔드포인트(/vision/product/confirm) 신설로 이어짐.

문구 편집 — 서버 호출 없이 실시간 미리보기

문구 위치를 9분할 고정 프리셋으로 돌지 자유 좌표로 돌지가 갈림길이었음. 이미지마다 여백이 달라 고정 위치는 제약이 컸으므로 **0~1 비율 자유 좌표**를 택함.

문제는 속도였음. 위치를 바꿀 때마다 서버에서 이미지를 재생성하면 매번 refine 생성 시간(약 12초)이 소요됨. 그래서 **드래그 중에는 서버를 부르지 않고 프론트 CSS로만 미리보기**하고, [완성하기] 시점에 최종 좌표를 담아 refine을 1회만 호출하는 구조로 설계함.

이 과정에서 base64를 URL 방식으로 바꾸자는 논의도 있었으나, **속도 병목의 실제 원인이 아님을 확인**하고 base64를 유지함. 트레이드오프는 CSS 미리보기와 서버 렌더링의 폰트 엔진이 달라 미세하게 어긋날 수 있다는 점이며, 미리보기 폰트를 서버 폰트와 맞추고 안전 여백을 뒤 완화함.

규제 표시 — safe 필드 기준 진행 판정

다음 화면 진행 여부를 block과 warn 둘 다 차단할지, block만 차단할지가 결정 지점이었음. **block은 명백한 법 위반이라 차단이 맞지만, warn은 경계선 표현이라 사용자가 인지한 채 진행할 수 있어야 서비스가 지나치게 깐깐해지지 않는다고 판단해 block만 차단함.** 서버가 safe 값을 계산해 내려주므로 프론트가 flags 를 순회할 필요도 없어짐.

3주차에는 프론트 mock의 규제 기준을 백엔드 regulation_rules.py 에 맞춰 통일함. mock끼리만 일치시키면 **프론트에서 확인한 동작과 실서버 동작이 달라져 연동 이후에 발견되는 버그**가 되기 때문임. mock의 목적은 실서버 동작의 예측이므로 기준점은 백엔드여야 함.

mock / real 이중 어댑터

copyApi.js · posterApi.js 에 real adapter를 추가하되 mock을 제거하지 않고 **환경변수로 전환**하게 유지함. GPU 서버나 VM이 불안정한 상황에서도 프론트 개발과 시연 흐름을 유지하기 위해서임. 실제로 VM 기동 실패(L4 재고 부족), 간헐적 500, GPU 필수라 로컬 실행 불가 등 서버 불안정 구간이 반복돼 이 결정은 3주 내내 값을 함.

트레이드오프는 **두 경로의 response shape와 필드명을 계속 맞춰 관리해야 한다**는 것이고, 실제로 이슈 2건의 원인이 됨.

배경 선택 UX

3주차에 배경 선택 페이지를 신설함. 기존에는 문구 선택 후 곧바로 시안 생성으로 들어가 **원하지 않는 AI 배경도 일단 생성**해야 함. 초기 안은 단색/그라데이션을 별도 화면으로 나누는 구조였는데, PM 리뷰로 AI 배경까지 포함한 **한 화면의 progressive disclosure**로 바뀜.

배경 형태: 단색 / 그라데이션
↓
색상 방식: 알아서 추천 / 직접 선택
↓ 직접 선택일 때만 펼침
색상 선택 영역 (단색 HEX 1개 / 그라데이션 HEX 2개)

API 전달 규칙도 함께 확정함 — 알아서 추천은 bg_colors 생략, 직접 선택은 background_mode + bg_colors .product의 category 는 전달하고, service의 academy / sports 는 포스터 API 계약에 없는 값이라 전달하지 않음.

브랜드와 완성도

서비스명을 AD메이커에서 **애드지니**로 바꾸고 보라/파랑/핑크 계열 브랜드 컬러와 마스코트를 적용함. 사이드바는 진행상황 → FAQ → 오늘의 상식 구조로 개편했고, 이력 저장 기능이 1차 범위에서 빠진 자리를 **"오늘의 광고 규제 한 줄"** 이라는 정적 순환 콘텐츠로 채워 서비스 정체성을 살림.

주요 이슈

이슈	원인	해결
문구를 가장자리로 끌면 제멋대로 줄바꿈	텍스트 박스가 <code>position: absolute + width: auto</code> 라 브라우저가 남은 공간으로 폭을 shrink-to-fit	<code>width: max-content</code> 로 고정 폭 부여, 여백 계산을 "절반 크기 + 지정 여백"으로 변경. 좌표 측정으로 검증
실API 연동 시 중복 요청과 500	React StrictMode 이중 실행 + 중복 클릭, 그리고 서버의 토큰라이저 동시성 문제가 겹침	<code>AbortController</code> + 버튼 비활성화로 프론트 원인 차단, 서버 로그를 함께 확인해 백엔드 원인을 분리
챗봇 강조점·추가요청이 이미지 생성에 반영 안 됨	mock과 real의 필드명 불일치 (<code>highlights ↔ keywords</code> , <code>extra ↔ request</code>)로 <code>buildPrompt()</code> 에서 누락	실제 API 계약 기준으로 필드 통일
포스터 API 실제 스키마가 스펙 문서와 다름	스펙이 초안 기준이었고 구현이 진행되며 필드가 구체화 됨	코드 리뷰로 차이를 정리해 담당자와 확인, 실제 구현 기준으로 스펙 문서 5장 전면 갱신 . mock도 실제 스키마에 맞춰 미리 구현
화면 전환 fade-in 미재생	React가 같은 위치의 div를 재사용(리마운트가 아니라 업데이트)해 애니메이션이 트리거되지 않음	상태별 <code><div></code> 에 서로 다른 <code>key</code> 를 부여해 새로 마운트
저장소를 새로 내려받으면 빌드 실패 위험	이미지 파일 전역 제외 규칙 때문에 마스코트·FAQ 스크린샷이 저장소에 포함되지 않음	프론트 자산 경로만 예외 허용

검증 방식

`npm run build / oxcint / git diff --check` 를 PR 필수 체크로 두고, Playwright로 홈 → 화면 A → B → C 전 구간을 클릭 검증함. 문구 모델은 신규 정적 IP 서버에서 `/vision/product · /suggest/options · /generate/copy · /validate/copy` 를 직접 호출해 실행작을 확인함. 챗봇 개편은 별도 독립 리뷰를 한 번 더 거쳐 Vision spec 유실, 동일 파일 재업로드, 빠른 중복 클릭, mock 데이터 정합성 문제를 발견하고 모두 수정함.

또한 팀원의 작업(문구 모델·규제 검증·챗봇·포스터·평가 모듈)을 지속적으로 리뷰하며 프론트 영향 범위를 사전에 정리함. 규제 등급 하향 판정 가능성, `bool("false")` 가 참으로 평가되는 함정, 후속질문 트리거의 한 글자 토큰 오탐은 모두 이 과정에서 나온 지적임.

미리보기와 최종 출력 정합

실서버 E2E를 돌리며 **미리보기와 최종 출력이 어긋나는 지점**을 차례로 잡은 구간임. 다섯 건의 공통 원인은 8-5절에 정리했고, 여기서는 각 건의 조치를 남김.

고품질화 단계에 제품 분류가 전달되지 않음 — drafts에만 있던 category 판정을 공용 `getPosterCategory()` 로 정리해 drafts와 refine이 같은 기준을 쓰게 함. product일 때만 전달하고 service의 `academy / sports` 는 포스터 API 계약에 없으므로 보내지 않음.

폰트 미리보기 불일치 — 실제 폰트 파일을 `@font-face` 로 로드하도록 고쳤는데 **개발 서버에서는 여전히 적용되지 않았음**.

```
The request id ".../poster_model/assets/fonts/Pretendard/Pretendard-Regular.ttf"
is outside of Vite serving allow list.
```

`src` 가 프로젝트 루트 밖을 가리켜 개발 서버가 차단하고 있었음. **빌드 환경에는 이 제한이 없어 빌드 확인만으로는 문제가 드러나지 않았고**, 개발 중 미리보기는 계속 대체 서체를 쓰고 있었음. `server.fs.allow` 추가로 해소함.

같은 PR에서 문구 외곽선도 맞춤. 미리보기는 `text-shadow` 로 아래쪽에 번지는 그림자였는데 서버는 `stroke_width=3` 으로 글자 둘레 전체에 외곽선을 그림. `-webkit-text-stroke` 로 바꾸고 **미리보기 캔버스 480px와 최종 출력 1024px 기준으로 환산함** ($3 \times 480/1024 \approx 1.4\text{px}$, 안팎으로 나뉘어 그려지므로 2.8px 적용). `paint-order: stroke fill` 을 넣지 않으면 안쪽 stroke 때문에 글자가 실제보다 가늘어 보임. 다만 브라우저와 PIL은 래스터라이저가 달라 안티앨리어싱까지 완전히 같게 맞출 수는 없다는 한계도 남김.

3:1 배너 문구 크기 — 3:1에서만 미리보기 문구가 최종 결과보다 크게 보였음. 프론트는 `canvasWidth`, 서버는 `min(W,H)` 를 기준으로 삼았는데 3:1에서만 짧은 변이 height라 어긋남. 3:1 전용 하드코딩 없이 프론트도 `min(width, height)` 기준으로 통일해 headline·sub·stroke 세 곳에 공통 적용함.

미리보기 줄바꿈 정합 — 문구 크기 기준을 맞춘 뒤 배너를 다시 확인하는 과정에서 **줄바꿈도 최종 출력과 어긋나 있다는 것**이 발견돼 두 건을 함께 다룸.

① 줄바꿈 불일치(버그) — 프론트는 **글자 수 상한**으로, 서버는 **실제 렌더링 폭**으로 줄을 나눔. 기존 글자 수 상한이 실제 폭보다 보수적이라 미리보기가 서버보다 문구를 더 잘게 쪼개고 있었음. 3:1뿐 아니라 1:1에도 차이가 있었지만 정사각형에서는 차이가 작아 눈에 띄지 않았음. 프론트도 실제 폭을 계산하도록 바꿈.

사용 가능 폭 = 캔버스 폭 - 2 × (짧은 변 × 0.06)
글자 크기(px) = 짧은 변 × size

서버와 동일한 폰트 파일을 이미 불러오고 있어 같은 기준으로 폭을 잴 수 있다는 점이 전제가 됨. 폰트 로드 전에는 대체 서체로 측정될 수 있어 **웹폰트 로드 완료 후 한 번 더 계산**함. 동작 변화도 함께 남김 — 기존에는 폰트 크기를 바꿔도 줄바꿈 지점이 유지됐지만, 이제는 서버와 동일하게 **글자 크기에 따라 줄바꿈이 다시 계산**됨. 두 동작을 동시에 만족시킬 수 없어 미리보기와 최종 결과의 일치 쪽을 택함.

② 배너 문구 크기(정책) — 기존 프리셋은 정사각형 기준이라, 서버가 짧은 변에 곱하는 3:1에서는 가장 큰 프리셋도 ****배너 가로 폭의 2.8%****에 그침. 서버 출력 기준으로 후보를 비교함.

후보	배너 폭 대비	긴 문구(20자)
현행	2.8%	한 줄
채택	5.3%	한 줄
후보 A	6.6%	두 줄로 쌓임
후보 B	8.0%	자동 축소 발생

자동 축소가 나면 미리보기와 다시 어긋나므로 제외했고, 배너는 한 줄로 읽히는 편이 자연스러워 5.3%를 채택함. **3:1에서만 별도 프리셋을 쓰고 1:1·3:4는 기존 값을 유지**함. 서버의 계산 기준 자체를 바꾸는 안도 검토했으나, 문구 좌표·여백·안전선·회귀 테스트가 모두 짧은 변 기준을 전제하고 있어 영향 범위가 컸음. **프론트도 결국 같은 기준으로 맞춰야 하므로 결과는 같고 수정 범위만 커짐**고 보고 제외함. 서버 변경은 없음.

검증 — 서체 5종 × 크기 3단계 × 비율 3종 × 문구 길이 3종을 교차해 확인했고 모든 경우에서 미리보기 줄 수와 실제 출력 줄 수가 일치함. 미리보기는 브라우저에 실제 로드된 웹폰트로 텍스트 폭을 측정하고, 최종 출력은 서버 생성 이미지에서 **글자가 존재하는 행을 기준으로 줄 수를 센** 방식임.

5-4. 서버 인프라 (정진우 · 하태진)

서버 조사를 인프라 플랫폼 / 외부 노출 계층 / 모델 실행 계층 3개 층위로 나눔.

인프라 플랫폼 — VM 단독

Cloud Run 콜드 스타트를 4단계(프로비저닝 5초 / 이미지 스트리밍 1~2초 / 엔진 초기화 5~15초 / **모델 로딩·VRAM 전송 = 실질 병목**)로 분해하고, 모델 저장 위치 4가지 비교로 **GCS + 병렬 다운로드가 가장 빠름**을 정리함.

결론은 VM 단독 사용. Cloud Run은 과금이 발생하는 반면 제공받은 VM은 무료이고, SD1.5+SDXL 약 11GB 적재로 콜드 스타트 손해가 큼.

이 과정에서 "**Cloud Run과 VM을 중첩해 쓸 수 있다**"는 잘못된 이해가 있었고 정정함. Cloud Run은 관리형 컨테이너 실행 환경이라 VM과 **병렬 선택지이지 계층 관계가 아님**. 조사 결과는 폐기하지 않고 기술 선택의 비교 근거로 재배치함.

외부 노출 계층

전제: CORS 사용, 긴 응답(refine), 큰 페이로드(base64), 추가 과금 발생 시 탈락.

방식	과금	긴 응답	큰 페이로드	판단
Nginx	없음	직접 조정	직접 조정	1순위
Caddy	없음	조정 가능	조정 가능	대안 (도메인 확보 시)
Cloudflare Tunnel	없음	120초 고정	쓰기 30초 고정	조건부
Tailscale Funnel	없음	제한 비공개	제한 비공개	임시 백업
Traefik / Ngrok / GCP ALB·API GW	—	—	—	배제

Nginx가 1순위인 이유 — 우리 조건에서 결정적인 변수는 refine의 긴 응답(15~50초)과 base64 왕복인데, **이 두 값의 상한을 우리가 직접 정할 수 있는 방식은 VM 내부 프록시뿐임**. 외부 터널은 사업자 고정 한도에 걸리고, 관리형 클라우드든 과금이 발생함.

만드시 손봐야 할 두 값도 미리 특정함 — `client_max_body_size` (기본 1MB → 413), `proxy_read_timeout` (기본 60초 → 504). **A안 내부의 분기점은 "도메인 확보 여부"**이며, IP로 가면 IP 인증서(2025년 7월부터 발급, **유효기간 6일**)의 갱신 자동화가 전제됨. 프로젝트 기간 내 구현이 어렵다고 판단해 실제로 적용하지는 않았음.

모델 실행 계층 — FastAPI 직접 호출

	FastAPI	TorchServe	Triton
최대 약점	동적 배치 없음, GIL 제약	프로젝트 사실상 종료	컨테이너 무거움, diffusion처럼 요청당 실행이 긴 워크로드에서 배치 이득 희박
판단	권장	비권장	조건부

모델 수가 적고 전후처리(rembg 마스크, 문구 합성, 워터마크)가 복잡해 FastAPI의 자유도가 가장 잘 맞음. 고도화 수단으로 검토했던 Triton은 변경점 과다와 담당자 일정으로 제외함.

실제 배포와 VRAM 실측

`final-project` 인스턴스(us-central1-a, g2-standard-4 + L4), `systemd` `poster.service`, 방화벽 `allow-model-api` 로 8000/8001 개방, 정적 IP 승격까지 완료. Nginx 실구성과 IP 인증서는 미착수임.

tmux → systemd 전환 작업을 수행함. 모델 11GB 다운로드 중 SSH 끊김을 막으려 tmux를 도입하고 서버도 tmux에서 띄웠는데 그럼에도 서버가 한 번 다운됨(OOM 아님). tmux는 SSH 끊김만 막고 프로세스 종료는 못 막는다는 것이 확인돼 systemd로 옮김. 이후 재시작 시 자동 기동이 안 됐는데, 원인은 **등록(파일 생성)과 부팅 시 자동 시작(enable)이 별개 절차**라는 점을 놓친 것이었음.

정적 IP 승격도 같은 성격의 작업이었음. VM을 중지하면 임시 외부 IP가 반납되어 재시작마다 주소가 바뀌고, 그때마다 프론트의 API 주소를 고쳐야 함. IP 인증서를 붙이는 경로도 주소가 고정돼야 성립함.

구간	피크	소요
1차 이벤트	2,952 MiB	약 19초 (GPU 100% 13초)
2차 이벤트	7,864 MiB	약 51초
종료 후	280 MiB (해제 확인)	—

전체 최대는 **L4 23GB 중 약 34%**. 주목할 점은 **2차 구간 대부분이 GPU 사용률 0~2%에 머물며 메모리만 증가**한다는 것임. 시간의 대부분이 연산이 아니라 **CPU offload 가중치 전송**이라는 뜻이며, 이후 GPU 상주 방식으로 전환하는 실측 근거가 됨.

5-5. PM (정진우)

범위 확정, 협업 체계 구축, 코드 리뷰, 파트 간 인터페이스 조율, 서빙 인프라 보조를 담당함.

- **1주차** (코드 리뷰 8건) — 대주제만 주어진 상태에서 타깃 업종 3종 · 기능 2종 · 인터페이스 형태를 확정해 팀원 4명이 각자 착수할 기준을 세팅. 저장소·협업 일지 템플릿·보고서 양식·GCP VM 구축. **폰트 3종 → 18종 확충** 및 fontTools 검증
- **2주차** (코드 리뷰 7건) — 서빙 3개 층위 조사 종합, VM 기동·방화벽·systemd·정적 IP·VRAM 실측, 팀원별 배포 안내서 작성 (모델 담당자는 모델에 집중하고 서버 기동은 PM이 전담하는 분담 확립), 500 에러 원인 규명, CLIP 77토큰 초과 발견, 고도화 방향 정리
- **3주차** (코드 리뷰 13건) — 협업 일지 통합본 4건, Design Planner 규제 우회 리스크 제거, 문구 모델 작업 축을 규제 정확도 → 카피 매력도로 전환 요청, text2img 범위 12종 → 1종 축소(PM 판단 착오 정정), 통합 문서 최신화

폰트 검증 결과 — fontTools로 18개 ttf를 측정해 한글 11,172자 전수 지원 8종 / 완성형(2,350~2,781자)만 지원 6종으로 등급을 나눔. 특이사항으로 재민3.0은 한자 8,137자를 가졌으나 **가나 0자**라 일본어 렌더링이 불가하고, 우아한세리프는 **영문 52자 전부 미지원**이라 순한글 전용임. 완성형 한정 폰트를 별도 목록화한 목적은 **희귀 음절에서 렌더링이 깨질 폰트를 사전에 식별**하는 것임. 최종 노출은 5종으로 좁힘.

6. 개발 과정과 의사결정

프로젝트 기간에 내린 결정 중, 후보를 놓고 비교한 뒤 트레이드오프까지 기록으로 남은 32건임.

6-1. 의사결정 — 범위와 정체성

항목	결정	선택 이유	트레이드오프
1	규제 필터링을 핵심 차별점 으로 (부가 기능 아님)	소상공인은 법무 인력이 없고 푸드·뷰티는 규제가 심함. LLM은 매력적인 카피(=대개 규제 대상 표현)를 잘 만들어 오히려 위험	룰 유지보수 부담(법령 개정 추적)
2	block / warn 2단계 판정 + 허용 표현 통과	과잉 차단은 도구를 못 쓰게 만들. "주름 제거"는 막되 "주 름 개선에 도움을 주는"은 통과해야 함. 대체 표현까지 제안 해 " 고쳐주는 도구 " 로	판정 로직·화이트리스트 복잡도 증가
3	챗봇 메인형 인터페이스	<code>/suggest/options</code> 가 고정 순차 질문 구조라 서브 보조로는 쓸 수 없음. 두 버전 목업 비교에서 메인형이 더 간 결	여러 선택지를 한 번에 훑는 경험 감소
4	게이트웨이 없이 프론트가 모델 API 직접 호출	1차 서비스 형태를 빠르게 완성하는 것이 우선	인증·라우팅·요청 통제 지점 부재
5	이력 저장 1차 제외 , base64 유지	URL 전환은 스토리지·DB 스키마·업로드/조회 로직을 새로 구축하는 작업	이력·공유·검색 미제공 → 사이드바를 "오늘의 광고 규제" 콘텐츠로 대체
6	문구는 LoRA 대신 프롬프트 엔지 니어링	이미지는 제품 보존 때문에 LoRA가 필요하지만, 텍스트는 데이터 수집·학습 시간 대비 개선폭이 작음	브랜드 톤을 학습으로 고정 불가
7	다양성 임계값(0.25)을 감이 아니 라 측정 으로	라벨링 21쌍의 유사도 분포(다른 시안 0~0.12 / 같은 시안 0.30~0.70) 사이값이라 근거를 설명할 수 있음	사전 라벨링 수작업

6-2. 의사결정 — 인프라와 신뢰도

항목	결정	선택 이유	트레이드오프
8	인프라는 GCP VM 단독 (Cloud Run 배제)	Cloud Run은 과금 발생, VM은 무료. 11GB 적재로 콜드 스타트 손해가 큼	오토스케일링·무중단 배포 포기
9	외부 노출은 Nginx 1순위	응답이 15~50초로 길어 타임아웃과 바디 크기를 직접 조정 할 수 있어야 함	도메인 확보 시 Caddy가 더 간단. IP로 가면 IP 인증서(6일) 갱신 자동화 부담
10	서빙 프레임워크는 FastAPI 직접 호출	모델 수가 적고 전후처리가 복잡해 자유도가 가장 잘 맞음. TorchServe는 프로젝트 종료, Triton은 diffusion에서 배치 이득 희박	동적 배치·모델 관리 포기
11	프로세스 관리를 tmux → systemd	tmux는 SSH 끊김만 막고 프로세스 종료는 복구 못 함. 실제로 한 번 다운됨	없음 (tmux는 다운로드 보호로 병행)
12	외부 IP를 정적 IP로 고정	VM 중지 시 임시 IP는 반납되어 재시작마다 주소가 바뀜 → 프론트 API 주소를 매번 수정해야 하고 IP 인증서도 재발급 필요	VM이 꺼져 있는 동안 소액 과금. 프로젝트 종료 시 주소 릴리스 필요
13	text2img 도입 + 무형 서비스업 확장	서비스가 "제품 사진이 있다"는 전제에 묶여 있어 이 전제를 푸는 것이 확장성 임팩트가 큼	가상제품·fake text 문제가 새로 발생. 업종 2개로 범위 제한
14	추가 업종은 학원 / 체육관·도장	규제 필터가 차별점이므로 규제가 강한 업종일수록 필터의 가치가 부각됨 . 근거법이 명확하고 위반 형태가 정형적	성형외과는 의료광고 사전심의 필요해 제외
15	하이브리드 신뢰 정책을 단조 (monotonic) 로	기존 하향 가드가 근거 문자열의 존재 여부만 보는 형식 검사 라 사실상 하향을 못 막고 있었음. 내용 검증 대신 하향 전면 차단으로 정리	LLM이 정당하게 판단한 오답 완화 기회 포기
16	Vision 결과를 사용자 확인 후 확정	제품명이 잘못 인식되면 강조점·문구·포스터까지 연쇄 영향	클릭 한 번 추가
17	3:1 내부 생성 해상도 1728×576	직접 생성은 서비스에 쓸 수 없는 수준, 2304×768도 12초 ↔150초로 편차가 큼. 1728×576은 4초 내외로 일정 . 제품은 최종 해상도 원본으로 재합성하므로 선명도 영향 없음	미세 질감 배경이 부드러워짐
18	3:4 AI 배경 보류	투명 제품에서 마스크 바깥 구조 생성. 배치 조정으로 해결 안 됨. 위험 제품을 자동 구분할 방법이 없어 열 수 없다고 판단	상세페이지에서 AI 배경 불가
19	기존 1:1 경로는 공통화하지 않고 분기 추가	AI 경로의 픽셀 단위 GPU 회귀 테스트를 자동화하기 어려움. 검증할 수 없는 변경보다 중복을 감수하는 편이 안전	코드 중복
20	로딩 체크리스트는 시간 기반 연출 유지 + elapsed 평균 보정	서버가 단계별 진행률을 내보내지 않는 구조라 실연동하려면 스트리밍 또는 폴링 엔드포인트를 새로 만들어야 함	실제 진행과 어긋날 수 있음. meta.elapsed 평균값으로 오차를 줄이는 선에서 타협

6-3. 의사결정 — 통합과 범위 조정

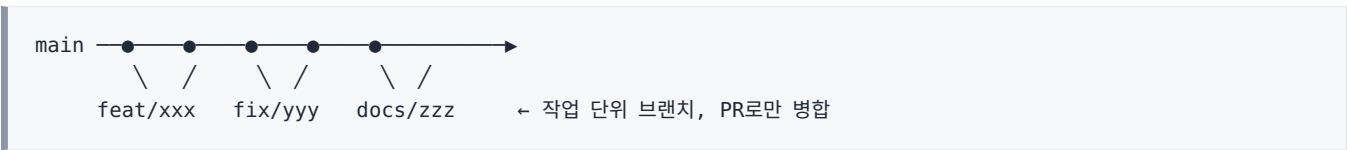
항목	결정	선택 이유	트레이드오프
21	Planner의 모든 문구 블록은 사용자 입력 또는 <code>/generate/</code> <code>copy</code> 결과에만 근거	규제 검증을 거치지 않은 문구가 포스터에 실리면 차별점 자체가 무너짐 . <code>derived</code> 는 생성 로직도 확인 불가	챗봇이 수집하지 않는 블록을 채울 수 없게 됨 → 다음 항목으로 대응
22	benefit/token/extra는 강조점 후속질문 으로 유도	고정 필드를 늘리면 모든 사용자가 불필요한 단계를 거침. 강조점 점은 이미 대화 맥락으로 생성되므로 필요한 사실값만 되묻는 것이 효율적	챗봇 단계 1개 증가 → 조건부 노출로 상쇄
23	Planner를 기존 흐름 대체가 아니라 3단계로 나눠 적용	대체안은 이미 연동이 끝난 API 계약과 UI를 제거하는 방향이라 리스크가 큼 . 사용자가 문구 위치·폰트를 지정하는 단계는 유지 가치가 있음	Planner 기능을 전부 쓰지 못함. 기존보다 낫다는 근거가 아직 없어 1:1 고도화 결과와 비교가 선행
24	배경 Vision을 <code>spec.product</code> 가 아닌 <code>background_context</code> 로 분리	product는 사용자 확정값 전용 슬롯 이라 배경 특징이 섞이면 규제 검증과 문구 근거가 오염됨	필드·엔드포인트 증가
25	문구 작업 축을 규제 정확도 → 카피 매력도 로 전환	안정성에만 매몰되면 안전하되 매력 없는 카피 가 나옴. 단 규제 통과율을 함께 측정해 위반율이 오르는 변형은 탈락 시키는 구조	규제 쪽 추가 발견은 문서로만 정리
26	text2img 범위를 12종 → 1종 완성형 으로 축소	동결 5일 전에 12종을 모두 만들면 어느 것도 완성도를 갖추지 못함	범위 축소 → 나머지는 고도화 전략으로. 범위 산정 착오는 PM이 회의에서 직접 정정
27	코드 동결을 8/22 → 8/25로 연기	진행 중 작업이 중단할 수 없는 상태였고 22일 동결 시 E2E 검증 시간이 확보되지 않음	팀원 부담 증가. 하루도 빠짐없이 작업이 이어져 피로가 누적된 상태 임을 공유하고 마지막 주를 마무리 작업 위주로 재배치
28	서빙 고도화(Triton) 제외	변경점 과다 + 담당자 일정. diffusion에서 배치 이득도 크지 않음	서빙 파트 깊이 감소 → "선택하지 않은 근거"를 기록으로 보완
29	프론트 mock 규제 기준을 백엔드 기준에 맞춤	mock끼리만 일치시키면 연동 이후에 발견되는 버그 가 됨. mock의 목적은 실서버 동작 예측	백엔드 를 변경 시 mock 동기화 필요
30	3:1은 좌측 카피 패널 + 우측 이미지(67%)	설계 단계 우려와 달리 6장을 실제로 합성해 확인 했더니 결합이 아니었음. 배너 비율이라 좌 카피/우 이미지가 관습적으로 읽힘	1:1:3:4와 계열이 다름
31	SDXL img2img를 고품질화 수단으로 확정하지 않음	12개 조건 중 구조 보존과 선명도 향상을 동시에 달성한 사례 0건 . 재실행 결과가 일치해 샘플링 요동이 아님도 확인	draft→refine 구조는 유지, refine 수단만 다시 여는 상태
32	구조 조건은 canny가 아니라 depth 우선	목표가 윤곽 복사가 아니라 구조 유지. draft의 윤곽이 왜곡된 경우 canny로 주면 SDXL이 왜곡까지 고착	depth만으로 업종 세부가 안 지켜질 수 있음

6-4. 역할 분담과 운영 원칙

5명이 파트를 나눠 병렬로 진행하되, **파트 경계에서 생기는 비용을 PM이 흡수**하는 방식으로 운영함. 대표적인 것이 배포 분담임. 모델 담당자가 각자 서버를 띄우면 GPU 환경 세팅에 시간이 분산되므로, **모델 담당자는 모델에 집중하고 VM 기동·방화벽·systemd는 PM이 전담**하는 구조를 2주차에 확립하고 팀원별 배포 안내서를 작성함.

6-5. Git 브랜치 전략

main 단일 보호 브랜치 + **작업 단위 토픽 브랜치** 구조임. 직접 push 없이 전부 PR을 거침.



항목	규칙	실측
브랜치 네이밍	<type>/<작업-슬러그>	원격 브랜치 37개 — feat 23 / fix 9 / docs 2 / chore 2
병합 방식	PR merge commit (squash 미사용)	main 커밋 220건 중 merge commit 64건 = PR 1건당 1개
커밋 메시지	<type>(<scope>) : 한국어 요약	fix 41 / feat 38 / docs 29 / frontend 27 / test 5 / chore 3 / perf 2
병합 조건	다른 팀원 1인 이상 승인	초기 테스트용 1건을 제외하고 전부 리뷰를 거쳐 병합

커밋을 압축(squash)하지 않은 이유는 커밋 단위 이력이 그대로 남아야 했기 때문임. 실제로 인물 배제 작업의 근거가 구버전 프롬프트였다는 것을 확인할 때, 어떤 시점에 어떤 프롬프트 문자열이 들어 있었는지를 이력에서 되짚어야 함. 매력 문구 A/B 실험 코드도 **삭제된 코드를 이력에서 복원**한 사례임. 이력이 압축돼 있었다면 두 작업 모두 어려웠음.

6-6. 협업 일지와 주간 보고서

프로젝트 첫날 `.github/DISCUSSION_TEMPLATE/daily-log.yml` 을 만들고 두 카테고리로 운영함. 팀원 각자가 쓰는 **Daily log**(5개 항목, 진행률 % 표기)와, PM이 이를 **파트별로 재구성**한 **Total daily log**임.

8/28 기준 **총 82건 이상**이 쌓였음 — 개인 Daily log 65건 이상, PM Total daily log 17건. 통합본은 개인 로그를 모으는 것이 아니라 **문구 모델 / 프론트엔드 / 이미지 생성 모델** 파트별로 재구성하고 "내일 할 일"을 합쳐 다음 날 우선순위를 만들.

마감 구간의 통합본은 특히 **파트 간 원인 이관 경로**로 쓰였음. 프론트가 확인한 "학원·체육관이 제품형처럼 생성됨"과 "문구 요청이 최대 2.6분 동안 응답 없이 대기함"이 그대로 이미지 모델의 파라미터 배선 작업과 문구 모델의 타임아웃 처리로 이어짐. **한 사람이 관측한 증상이 다른 파트의 작업 항목이 되는 흐름**이 문서로 남아 있음.

주간 보고서의 목적은 팀원 간 작업 공유와 **취업 시 본인 담당 작업·판단 근거를 설명할 자료** 두 가지였음.

0. 기본 정보
1. 이번 주 작업 내용
2. 주요 이슈와 해결 (문제-원인-해결-결과)
3. 주요 결정 사항과 이유 (후보-선택 이유-트레이드오프)
4. 다음 주 계획
5. 막힌 것

"후보 - 선택 이유 - 트레이드오프" 구조를 강제한 것이 이 프로젝트 기록의 질을 결정함. 결정만 적으면 "왜"가 사라지고, 트레이드오프를 적게 하면 "무엇을 포기했는지"까지 남음. 본 보고서 6-1~6-3절이 이 양식 위에서 만들어짐.

6-7. PR 리뷰 문화

PR은 다른 팀원의 Approve 후 병합함. 단순 승인이 아니라 코드·문서·화면을 직접 확인하고 질문을 남기는 방식이었고, 파트 간 교차 리뷰가 일상적이었음 — 프론트 담당이 백엔드 규제 로직을, 백엔드 담당이 프론트 mock을, 포스터 담당이 프론트 API 요청 필드를 리뷰함.

7. 결과 및 평가

7-1. 규제 검증 성능

지표	초기	중간	3주차	최종 (main, 8/28)
룰 개수	13	49	고유 50	고유 50
카테고리	3	3	5	5
block 보유 카테고리	2/3	4/5	5/5	5/5
골드셋 케이스	40	81	118	127 (train 71 / holdout 43 / redteam 13)
TRAIN 정확일치	—	100%	100%	100%
HOLDOUT 정확일치	—	90.6%	93.0%	93.0%
대표 정확일치	—	96.3%	97.1%	97.4% (n=114)
대표 탐지 F1	85.7%	96.7%	97.6%	97.7%

대표값을 HOLDOUT으로 잡는 이유 — TRAIN은 룰을 고치면서 함께 본 셋이라 100%가 나오는 것이 순환 논리임. 대표 수치를 전체 96.3%에서 HOLDOUT 90.6%로 **낮추는 방향으로** 재정의한 것이 3주차 리뷰의 결과임.

매력도 A/B 실측 — persona 승률 0.708 / hook 0.667 / cta 0.583. 셋 다 위반율 0을 유지했고 persona가 재측정에서도 일관되게 우세함.

7-2. 이미지 생성 성능

항목	값
draft / refine	18.28초 / 53.22초 (text2img), refine 12~16초 (일반)
3:1 내부 생성	3.7~4.2초
서비스형 시안 (파라미터 분리 후)	3.4초 → 1.7초 (해상도 768 → 512)
/compose/text	CPU 수백 ms
VRAM — CPU offload 시절	피크 7,864 MiB, 해제 후 280 MiB (호스트 RAM 10.2GB 점유 → OOM)
VRAM — VRAM 상주 전환 후	21,402 MiB 정상 상태 , refine 6회 교대 6/6 통과 , 전환 48.7 → 9.9초
동시 요청	text2img 3건 동시 — HTTP 200 1 → 3 , 500 2 → 0
프롬프트 길이	90 → 70 token (한도 77)
폰트	검증 18종 → 노출 5종

VRAM 수치가 7,864 → 21,402 MiB로 크게 오른 것은 누수가 아니라 **모델 상주 위치를 호스트 RAM에서 VRAM으로 옮긴 결과**임. 이 장비는 VRAM 24GB / RAM 16GB라 CPU offload가 오히려 불리함.

7-3. 테스트 및 검증 체계

기능별로 검증 수단을 다르게 둬. **자동화할 수 있는 것과 없는 것을 구분한 것이 핵심**임.

대상	수단	자동화	비고
규제 룰 · 하이브리드 판정	pytest + 골드셋 회귀 잠금	자동	테스트 함수 259개 중 다수
LLM 호출 경로	<code>_client_chat</code> patch 통합 테스트	자동	형식 검사가 아닌 실제 경로 흉내
프론트 빌드 · 린트	<code>npm run build /oxlint/ git diff --check</code>	자동	PR 필수 체크
사용자 흐름	Playwright (홈 → A → B → C)	자동	클릭 단위 검증
이미지 회귀	동일 입력·동일 시드 해시 비교	자동	파라미터 변경 시 제품형 회귀 확인에 사용
이미지 품질	육안 판정 + 선명도 지표 + 대조군	수동	픽셀 단위 GPU 회귀 자동화는 포기
레이아웃 안정성	그림체 24장 전수 합성 후 문구 축소 건수	수동	0건 확인
미리보기-출력 일치	서체 5 × 크기 3 × 비율 3 × 길이 3 교차	수동	줄 수 전수 일치

자동화하지 않기로 한 것을 명시함. AI 경로의 픽셀 단위 GPU 회귀 테스트는 비결정성 때문에 자동화가 어렵다고 보고, 기존 1:1 경로를 공통화하는 대신 **분기를 추가해 코드 중복을 감수함**. "검증할 수 없는 변경보다 중복이 안전하다"는 판단임.

7-4. 재현 검증 결과

최신 main을 직접 실행해 문서 수치를 재현함.

항목	문서값	재현값	판정
룰 개수 (food/beauty/goods/sports/academy)	20/18/20/15/13	동일	일치
block 개수	6/5/3/2/2	동일	일치
골드셋 구성	train 71 / holdout 43 / redteam 13	동일	일치
TRAIN / HOLDOUT 정확일치	100% / 93.0%	동일	일치
전체 대표 정확일치 / F1	97.4% / 97.7%	동일 (n=114)	일치

골드셋을 확충하면서 홀드아웃을 건드리지 않은 것이 중요함. 신규 9건을 전부 train에 넣고 holdout 43건은 원본 그대로 보존함. "내가 만든 쉬운 사례를 미노출셋에 넣어 점수를 부풀렸다"는 오해를 원천 차단하기 위한 조치이며, HOLDOUT 93.0%가 편입 전후로 변하지 않는 것이 그 증거임.

8. 문제 해결 과정

프로젝트 기간에 발생한 이슈 중 파급이 컸던 것을 추림. 모두 **문제** → **원인** → **해결** → **결과** 순서로 정리함.

8-1. 규제 검증

① **A/B 실험이 규칙 유무와 무관하게 위반율 0%** 문제 규제 지침을 준 그룹과 안 준 그룹의 위반율이 둘 다 0%로 나와 효과가 측정 되지 않음. 원인 규칙이 무효한 것이 아니라 **입력이 순해서 모델이 위반할 이유가 없었던 실험 설계 결함**. 해결 "소상공인이 위반 표현을 직접 요청하는 상황"으로 입력 세트를 재설계. 결과 위반 1→0건, 경고 7→3건. **데이터를 왜곡하지 않고 설계를 고쳐 효과를 입증함**.

② **규제 필터 성능지표가 실제보다 높게 보고됨** 문제 precision 100% / recall 97.8%로 **지나치게 높은 수치**. 원인 평가 러너가 3등급을 탐지 여부의 **이진값으로 축약**해, block으로 잡아야 할 것을 warn으로 판정한 등급 오류가 지표에 반영되지 않음. 해결 1차 지표를 **3-클래스 라벨 정확일치율**로 교체, `under_severity` 를 양방향 검출로 수정, 경계 정상 문구 13개를 오탐 스트레스 테스트로 편입. 결과 재측정 전체 96.3% / HOLDOUT 90.6%로 **실제 성능이 드러남**. 대표 수치도 HOLDOUT으로 재정의하고 TRAIN 100%가 순환임을 문서에 명시함.

강사 피드백("성능지표가 너무 높아 오히려 의심됨")과 PM 코드 리뷰(이진화에서 등급 구분이 사라지는 지점 지적)가 같은 시점에 맞물려 발견됨. 재측정 결과 **이번 수치가 이 버그로 부풀려지지는 않았음**을 확인했지만, 지표가 등급 오류를 못 잡는 구조는 그대로였기 때문에 고침.

③ **커버리지를 케이스 수 / 룰 수로만 측정하던 한계** 지표가 '룰이 몇 개인가'만 보고 '**어떤 축에 룰이 있는가**'를 보지 않았음. rule coverage matrix로 전환하자 goods block 0건, sports 의료행위 축 전무, academy 환불 축 공백이라는 **갭 3건이 드러났고** 순차 보강해 block 보유 카테고리 4/5 → **5/5**가 됨.

④ **후속질문 트리거의 한 글자 토큰 오탐** 트리거에 "원", "역", "분" 같은 한 글자가 있어 "**지원**", "**역동적**", "**분위기**", "**성분**"에도 후속질문이 발생함. 금액·시간처럼 숫자를 동반하는 사실값은 "**숫자+단위**" 정규식으로만 판정하도록 바꾸고 false-positive 테스트를 추가함.

8-2. 포스터 생성 모델

① **배경 품질 비교가 무효한 조건에서 이뤄지고 있었음** 문제 육안 차이가 거의 없어 "고품질화 효과가 크지 않다"고 판단. 원인 로그를 다시 보니 **refine 입력이 실제 draft가 아니라 단색 대체 이미지였음** — 검증 스크립트가 입력을 파일명 패턴으로 추측하다 실패하면 **조용히 단색 배경으로 대체**하고 있었음. 해결 입력을 케이스 ID로 직접 지정해 조용한 대체를 제거. 결과 실제 차이가 드러나 (미세 질감 328 대 53) **이전 판단을 뒤집음**.

"조용한 실패"가 실험 결론을 통째로 뒤집을 뻔한 사례임. 이후 API 설계에도 같은 원칙이 적용됨 — refine에서 명시값과 추론값이 다르면 **보정하지 않고 차단**함(fail-closed).

② **화질 개선 검증이 해상도 차이에 가려질 뻔함** 문제 시안 768 vs 결과 1024라 개선이 diffusion 덕인지 해상도 덕인지 구분되지 않음. 원인 refine 내부가 LANCZOS 업스케일 후 diffusion을 엮는 구조라 **업스케일이 사실상 refine의 입력**. 해결 업스케일만 한 대조군 추가. 결과 업스케일만으로도 선명도 지표가 크게 흔들려(2234→772) **대조군 없이는 판단 불가능**이었음이 드러났고, 대조군 기준으로는 4건 중 3건에서 선명도가 오히려 감소함. strength·실효 step까지 통일해 재확인해도 **12개 조건 중 구조 보존과 선명도 향상을 동시에 달성한 사례가 0건**이었음.

③ **production 결과를 좌측 가장자리 색 띠** draft 약 13px, refine 약 24px의 색 띠가 남음. VAE tiling을 첫 후보로 통제 실험했으나 ON/OFF 모두 0px로 나와 가설이 지지되지 않았음. **원인이 불명확한 상태라 production 설정을 바꾸지 않았고, 후처리로 잘라내 가리지도 않았음.** 이후 추가 분리 실험에서 SD1.5 VAE tiling이 원인으로 확인돼 3:1 draft에서만 no-tiling 인스턴스를 쓰도록 분리함. 증상만 가리지 않은 판단이 정확한 원인 특징으로 이어짐.

8-3. 서빙 인프라

① **프론트 연동 중 간헐적 500 오류 (4회 중 2회 실패)** 문제 동일 요청이 성공(16초, 15.2초)과 500을 오갔고 CORS 에러도 함께 표시됨. 원인 `RuntimeError: Already borrowed` — `CLIPTokenizerFast` 를 여러 스레드가 동시 사용할 때 발생하며 GPU 메모리 문제가 아니었음. **CORS 에러는 별개 원인이 아니라 500 응답에 CORS 헤더가 붙지 않아 함께 표시된 것.** 해결 pipe 사용 구간 전체를 락으로 직렬화. 결과 3건 동시 요청에서 HTTP 200 **1→3** / 500 **2→0**.

증상만 보면 CORS 설정 문제로 오진하기 쉬운 사례임. 후속 처리에서도 같은 태도가 이어짐 — 담당자는 예외를 내는 tokenizer만 잠그면 증상이 사라진다는 것을 알면서도, **예외를 내지 않고 조용히 잘못된 이미지를 만드는 scheduler 가변 상태까지** 함께 직렬화함.

② **프롬프트가 CLIP 토큰 한도를 초과해 뒷부분이 잘림** 문제 제품 보존 지시가 이미지에 반영되지 않음. 원인 95토큰으로 77토큰 한도를 초과했고 초과분이 **에러 없이 조용히 절삭됨.** 잘려나간 것은 `single product only, isolated product shot, no other objects` 등 제품 보존 지시였음. 해결 프롬프트 예산 모듈로 보호 영역·가변 영역을 분리. 결과 90 → 70 token. **파급:** 기존 디자인 실험에서 "모델이 지시를 잘 따르지 않는다"고 판단했던 결과 일부가 **truncation된 조건에서 측정된 것일 수 있어 재검증 대상**이 됨.

③ **refine 반복 시 OOM Kill** 문제 VM에서 refine을 반복하면 `poster.service` 가 종료됨(7회 중 2회). 원인 `enable_model_cpu_offload()` 가 가중치를 **호스트 RAM에 상주시키는** 방식인데, 이 장비는 **VRAM 24GB / RAM 16GB**라 반대로 동작함 — RAM 10.2GB 점유, VRAM은 280 MiB만 사용. 해결 상주 위치를 VRAM으로 전환하고 워밍업 대상과 상주 모델 수를 조정. 결과 refine 6회 교대 **6/6 통과**, 전환 시간 48.7 → 9.9초.

④ **VM 기동 실패 (L4 GPU 재고 부족)** `us-central1-a` 존의 일시적 가용량 부족. 재시도로 기동했으나 **구조적 해결이 아니어서 시연 당일 재발 가능함.** 시연 영상 사전 확보를 백업으로 준비함.

8-4. 통합 · 설계 (PM)

① **Design Planner가 규제 검증을 거치지 않은 문구를 생성할 수 있는** 구조 문제 Planner가 배치하는 문구 블록의 `source` 가 `user / generated / derived` 3종인데 **derived 는 생성 로직을 확인할 수 없었음.** 원인 Planner 계층이 문구 모델과 독립적으로 문구를 만들 수 있는 경로가 열려 있었음. 문구 모델을 거치지 않은 문구가 포스터에 실리면 **서비스의 핵심인 규제 검증이 우회됨.** 해결 모든 문구 블록을 **사용자 입력 또는 /generate/copy 결과에만 근거**하도록 제한하고, 사용자 입력 경로에도 즉시 규제 검사를 추가(4중 방어 ④). 결과 "포스터에 실리는 모든 문구는 규제 검증을 거친다"는 불변식이 구조로 고정됨.

문구 모델 담당자가 미리 정리해둔 **Claim-Locked Copy 설계와 동일한 문제**임이 확인됨. 서로 다른 파트에서 독립적으로 발견한 두 문제가 사실 같은 문제였던 사례임 — Planner 쪽에서는 "문구 블록의 출처를 어떻게 묶을 것인가", 문구 모델 쪽에서는 "LLM이 없는 사실을 지어내는 것을 어떻게 막을 것인가"로 각각 접근하고 있었음.

② **text2img 고도화 지원 범위를 과도하게 잡음** 분기 2종 × 배경 2종 × 비율 3종 = **12종을 코드 동결 5일 전에** 모두 포스터 수준으로 만들어야 하는 상황이 됨. 원인은 고도화 방향을 정할 때 **담당자의 실제 작업량을 충분히 반영하지 못한 PM 판단 착오**였음. 회의에서 범위 산정이 과했음을 직접 인정하고 1종 완성형으로 축소함.

- ③ 챗봇 0단계 분기 질문이 사용자 관점에서 직관적이지 않음 "제품이 있으신가요? 서비스 위주인가요?"는 개발 관점에서는 명확하지만 사용자에게는 한 번에 이해되지 않았다(카페 운영자는 커피 홍보와 매장 분위기 홍보 둘 다 해당). 결과물 기준 질문으로 바뀌어 "어떤 광고를 만들어 드릴까요?"로 변경했고, 용도 선택에 비율 표기도 추가함.
- ④ Cloud Run과 VM을 중첩해 쓸 수 있다고 잘못 이해함 두 서비스의 실행 모델을 혼동한 것으로, Cloud Run은 VM과 병렬 선택 지이지 계층 관계가 아님. 구조 재확인 후 정정 공유했고 VM 단독으로 확정함.

8-5. 프론트-서버 정합

미리보기와 최종 출력이 어긋나는 문제는 마감 구간에 다섯 번 나왔고, 원인이 전부 "두 쪽이 서로 다른 기준으로 계산하고 있었다"는 한 가지였음.

어긋난 축	프론트 기준	서버 기준	해결
제품 카테고리	초안 생성에서만 판정	고품질화는 기본값 goods	공용 판정 함수로 통일
서체	대체 서체 (개발 서버에서 차단 됨)	실제 폰트 파일	폰트 경로 접근 허용
외곽선	아래로 번지는 그림자	글자 둘레 전체에 외곽선	외곽선 방식으로 변경 + 캔버스 비율 환산
문구 크기	캔버스 가로 폭 기준	짧은 변 기준	프론트도 짧은 변 기준으로 통일
줄바꿈	글자 수 상한	실제 렌더링 폭	프론트도 실제 폭 계산

교훈은 계약 문서만으로는 부족하다는 것임. 필드명과 타입은 스펙 문서로 맞출 수 있지만, "어떤 변을 기준으로 곱하는가" 같은 계산 규칙은 스펙에 적히지 않은 채 양쪽이 각자 구현함. 이 다섯 건은 전부 실서버 E2E에서만 드러났고, mock과 빌드 확인으로는 한 건도 잡히지 않았음.

9. 한계점 및 향후 계획

9-1. 현재 한계

완성도가 아니라 **어디까지 검증됐는지**를 기준으로 정리함. 검증하지 않은 것을 통과로 적지 않는다는 원칙을 그대로 적용함.

영역	한계	성격
규제 커버리지	REDTEAM(의미 회피) 정확일치 23.1% — 룰 단독으로는 우회에 취약	구조적
규제 커버리지	갭 G4~G7 미해소 — academy 근거법 단일, whitelist 편중, 다국어 미지원, 실증 책임 미구조화	범위
규제 커버리지	룰이 정규식이라 맥락을 못 봄 — 오탐 2건("최고의 하루", "완벽한 오후")이 대표 사례	구조적
고품질화	SDXL img2img가 구조 보존과 선명도 향상을 동시에 달성한 사례 0/12 — refine 수단이 확정되지 않은 상태	구조적
이미지 범위	3:4 AI 배경 미지원 — 투명 제품에서 마스크 밖 구조 생성, 위험 제품 자동 판별 불가	기술적
이미지 검증	비정사각(3:1~3:4) + AI 배경 조합의 실서버 검증 미수행	검증 공백
입력 이미지	손과 제품이 함께 찍힌 사진은 rembg가 손을 제품 일부로 잡을 수 있어 최종 합성에 원본 손이 남음. 프롬프트로 제거할 수 없음	입력 한계
타이포	타이포 프리셋과 한국어 헤드라인 충돌 — 판면 4종 중 13자 삽입 시 2종, 최대 길이에서는 1종만 통과	기술적
인프라	Nginx 실구성·IP 인증서 미착수 — 현재 프론트가 모델 API를 직접 호출	미구현
인프라	게이트웨이 부재 → 인증·라우팅·요청 통제 지점 없음	설계 선택
인프라	<code>SINGLE_WORKER_ONLY = True</code> — worker를 늘리면 pipe Lock이 무력화됨	확장 제약
인프라	L4 GPU 존 재고 부족으로 VM 기동 실패가 시연 당일 재발 가능	외부 요인
기능	이력 저장·공유·검색 미제공, 회원 인증 없음	범위
실험	77-token truncation 조건에서 측정된 기존 디자인 실험 결과 일부가 재검증 대상	검증 공백
평가	매력도 A/B는 LLM 심사위원 판정이며 실제 소상공인 사용자 평가는 없음	평가 공백

마지막 항목을 특히 강조해 둠. 이 프로젝트에는 **실사용자 테스트가 없음**. 규제 판정은 사람이 라벨링한 골드셋으로 정직하게 짚지만, "이 광고가 실제로 매출에 도움이 되는가"는 측정하지 않았고 측정할 수단도 없었음.

9-2. 개선 방향

"왜 하지 않았는가"와 "다음에 무엇을 할 것인가"를 함께 정리함.

영역	방향	근거 / 선택하지 않은 이유
규제 검증	레드팀 확대 + 라우팅 정책 개선	병목이 LLM 성능이 아니라 라우팅 정책 임이 실측으로 확인됨(에스컬레이션되면 정확도가 크게 오름)

영역	방향	근거 / 선택하지 않은 이유
규제 검증	lexicon 기본 + broad 를 고보증 모드로 옵션화	현재는 정확도 우선으로 broad 고정. 비용 최적화는 운영 단계 과제
규제 검증	Claim-Lock 단계적 롤아웃	수집값의 출처 기록으로 검증 근거는 확보됨. 표시광고법 실증 책임 구조화로 연결
규제 검증	rule_stats 확장으로 커버리지 매트릭스 자동 생성	현재는 수작업. 룰이 늘수록 갭 탐지가 사람 손에 의존
규제 검증	다국어 규제 (번역본은 대상국 기준 으로 검증)	한국 법령 기준 룰을 번역문에 그대로 적용하면 틀림
이미지 생성	ControlNet Depth 로 조건부 재생성	canny는 draft의 왜곡된 윤곽까지 고착시킴. 목표가 윤곽 복사가 아니라 구조 유지
이미지 생성	결과에 따라 draft 모델을 SDXL 계열로 교체 검토	단 draft 3장 생성 시간이 사용자 대기에 직결되므로 속도 실측이 선행
이미지 생성	AI 배경 3:4 개방	위험 제품(투명 재질) 자동 판별이 전제. 판별 없이 열면 조용한 품질 저하
이미지 생성	Planner의 초기 디자인 품질 개선 (구성·정보 위계·제품과 문구 관계)	후처리 보정보다 처음에 어떤 디자인을 만드느냐 가 상위 문제로 확인됨
서빙 인프라	Nginx + IP 인증서 자동 갱신	조사·설계는 완료. 인증서 유효기간 6일이라 갱신 자동화가 필수 전제
서빙 인프라	Triton은 트래픽이 커지면 1순위	현재는 diffusion 워크로드에서 배치 이득이 희박하고 변경점이 과다
서빙 인프라	Cloud Run은 실서비스·과금 상황이면 유리	최적 구성(GCS 병렬 다운로드 + Direct VPC)까지 조사 완료. 무료 VM이 있는 현 조건에서만 불리
서빙 인프라	관리형 LB는 단일 인스턴스에서 오버스펙	존재 이유 대부분이 "백엔드가 여러 대일 때" 성립
서빙 인프라	job API + polling/SSE로 실시간 진행률	현재 로딩 체크리스트는 시간 기반 연출 + elapsed 평균 보정
서비스 확장	매력도 A/B의 프로덕션 반영	하네스와 1차 실측까지 완료 (persona 0.708). 반영 여부만 팀 논의 대기
서비스 확장	규제가 강한 업종 우선 확대 (세무·법무)	규제 필터가 차별점이므로 규제 강도가 곧 제품 가치
서비스 확장	이력·공유·검색	DB·스토리지 도입이 함께 필요. 1차에서 의도적으로 제외
서비스 확장	K-컬처 정체성 (SDXL 비교 · LoRA 검토)	그림체 실험이 truncation 조건에서 측정됐을 가능성이 있어 재검증이 선행

10. 프로젝트 회고

10-1. 기술적으로 배운 것

측정 설계가 측정값보다 먼저임. 이 프로젝트에서 판단을 뒤집은 사건은 대부분 모델이 아니라 **측정 방식**에서 나옴. 규제 A/B가 0%로 나온 것은 규칙이 무효해서가 아니라 입력이 순해서였고, 고품질화 효과가 안 보인 것은 refine 입력이 조용히 단색 이미지로 대체돼 있었기 때문이며, 성능지표가 지나치게 높았던 것은 3등급을 이진값으로 축약했기 때문임. 세 건 모두 **결과를 의심하지 않았다면 그대로 보고했을 수치**였음.

대조군이 없으면 "좋아졌다"는 말을 할 수 없음. 시안 768 vs 결과 1024를 그냥 비교하면 개선이 diffusion 덕인지 해상도 덕인지 알 수 없음. 업스케일만 한 대조군을 넣었더니 그것만으로도 지표가 크게 흔들렸고, 결론이 뒤집힘.

조용한 실패가 눈에 보이는 실패보다 위험함. Already borrowed 예외는 500으로 드러나지만, scheduler 가변 상태 공유는 예외 없이 잘못된 이미지를 만들. CLIP 77토큰 초과도 여러 없이 뒷부분을 잘라냄. 두 경우 모두 **예외를 내는 쪽만 고치면 증상은 사라지지만 안전해지지는 않는다**는 판단으로 범위를 넓혀 처리함.

계약 문서로 맞출 수 있는 것과 없는 것이 다름. 필드명과 타입은 스펙으로 통일할 수 있지만, "어떤 변을 기준으로 곱하는가", "줄바꿈을 글자 수로 세는가 렌더 쪽으로 세는가" 같은 계산 규칙은 스펙에 적히지 않은 채 양쪽이 각자 구현함. 마감 구간 정합 이슈 5건이 전부 이 유형이었고, mock과 빌드 확인으로는 한 건도 잡히지 않았음.

서버에 기능이 있어도 호출부에 배선이 없으면 없는 것임. 제품형과 서비스형을 구분하는 파라미터는 서버 분기와 테스트까지 다 갖춰져 있었지만, 프론트가 값을 보내지 않아 한 번도 적용되지 않았음. 기능 완료와 동작 확인은 별개임.

10-2. 협업에서 배운 것

"후보 - 선택 이유 - 트레이드오프" 양식을 강제한 것이 기록의 질을 결정함. 결정만 적으면 "왜"가 사라지고, 트레이드오프 칸을 비워 둘 수 없게 하면 "무엇을 포기했는지"까지 남음. 6장의 의사결정 32건은 전부 그 양식 위에서 3주간 축적된 것이지 마감에 몰아서 쓴 것이 아님.

교차 리뷰가 실제로 코드를 바꿈. 프론트 담당이 백엔드 규제 로직의 등급 하향 가능성을 잡아냈고, PM 리뷰가 평가 지표 자체를 바꿈. 특히 후자는 **대표 수치를 96.3%에서 90.6%로 낮추는 방향의 수정**이었고, 팀이 그것을 받아들였다는 점이 이 기록의 신뢰도를 만들.

자기 근거를 스스로 뒤집는 것이 가능한 팀이었음. 인물 배제 작업에서 담당자가 자기 작업의 근거가 구버전 프롬프트였음을 스스로 발견하고, "불필요하다고 보시면 전체를 되돌려도 좋다"고 열어둔 채 팀 의견을 묻은 사례가 있음. 되돌리는 비용을 개인이 감수하지 않아도 되는 분위기가 있어야 나올 수 있는 행동임.

증상을 관측한 사람과 고치는 사람이 다를 때, 문서가 이관 경로가 됨. 프론트가 관측한 "학원이 제품처럼 생성됨"과 "요청이 2.6분 동안 응답 없이 대기함"이 통합 일지를 거쳐 각각 이미지 모델과 문구 모델의 작업 항목이 됨. 구두로 넘겼다면 원인 규명 과정이 남지 않았을 것임.

10-3. 아쉬운 점

범위 산정에서 판단 착오가 있었음. text2img 고도화를 분기 2종 × 배경 2종 × 비율 3종 = 12종으로 잡았는데, 코드 동결 5일 전에 그 전부를 포스터 수준으로 만드는 것은 불가능한 양이었음. 원인은 방향을 정할 때 담당자의 실제 작업량을 충분히 반영하지 못한 것이고, 회의에서 직접 정정해 1종 완성형으로 축소함. **정정이 늦지 않은 것은 다행이지만, 애초에 담당자와 작업량을 먼저 확인했어야 할 판단이었음.**

코드 동결을 8/22에서 8/25로 한 번 연기함. 진행 중 작업이 중단할 수 없는 상태였고 22일 동결 시 E2E 검증 시간이 확보되지 않았음. 결과적으로 실서버에서만 드러나는 문제 12건을 잡을 수 있었지만, 대신 **하루도 빠짐없이 작업이 이어져 팀 피로가 누적됨.** 일정에 완충을 두지 않은 계획의 대가였음.

실사용자 테스트를 하지 못함. 규제 판정 정확도는 골드셋으로 정직하게 잴지만, 소상공인이 이 서비스를 실제로 써서 광고를 만드는 과정은 한 번도 관찰하지 못함. 챗봇 0단계 질문이 개발 관점 문구였다는 것도 사용자 테스트가 아니라 리뷰에서 발견됨.

서빙 고도화가 조사에서 멈춤. Nginx 구성과 IP 인증서, Triton 검토까지 조사와 설계는 끝냈지만 실제 적용은 하지 못함. "왜 선택하지 않았는가"를 근거로 남긴 것은 의미가 있으나, 조사와 구현 사이의 간극은 남음.

10-4. 다음에 적용할 것

- 범위를 정하기 전에 담당자와 작업량을 먼저 맞춤. 방향은 PM이 정할 수 있어도 소요는 담당자가 앎.
- 일정에 검증 구간을 처음부터 확보함. 이번에는 동결을 연기해서 만들었지만, 처음부터 E2E 검증 시간을 일정에 포함시켜야 함.
- 파트 간 계산 규칙까지 스펙에 적음. 필드명·타입뿐 아니라 "기준이 되는 변", "단위", "반올림 방향"처럼 양쪽이 각자 구현하게 되는 규칙을 명시함.
- 평가 지표를 만들 때 "이 지표가 못 잡는 오류는 무엇인가"를 먼저 적음. 이진화가 등급 오류를 못 잡는다는 것을 지표 설계 시점에 알았다면 재측정이 필요 없었음.
- 사용자에게 보여주는 흐름은 개발 관점 언어가 아닌지 초기에 확인함.

11. 참고자료

11-1. 근거 법령 및 규제 자료

규제 룰 사전 50개는 아래 법령과 사례를 근거로 작성함. 룰마다 근거 법령을 필드로 붙여 판정 결과에 함께 반환함.

구분	근거	적용 지점
공통	표시·광고의 공정화에 관한 법률 (표시광고법)	COMMON 룰 9개 — 부당 표시·광고, 실증 책임
식품	식품 등의 표시·광고에 관한 법률	food 카테고리 — 질병 예방·치료 표방 금지
식품	건강기능식품에 관한 법률	food 카테고리 — 기능성 표시 제한
화장품	화장품법 제13조	beauty 카테고리 — 의약품 오인 광고 금지
의약·의료기기	약사법 제61조 / 의료기기법 제26조	goods 카테고리 — 의약품·의료기기 오인 (block)
상표	상표법	goods 카테고리 — 위조품·상표 침해 (향후 과제)
체육	국민체육진흥법 / 의료법	sports 카테고리 — 의료행위 오인
방문판매	방문판매 등에 관한 법률	sports 카테고리 — 계약 해지·환불
학원	학원의 설립·운영 및 과외교습에 관한 법률 시행령	academy 카테고리 — 수강료 반환 기준
사례	공정거래위원회 제재 사례	academy 카테고리 — 과장 광고 판단 기준
AI 고지	인공지능 발전과 신뢰 기반 조성 등에 관한 기본법 제31조	AI 생성물 표시 3지점

법령 원문 정리와 출처는 저장소 docs/ 아래에 별도 문서로 남김 — 광고규제_법규정리.md , 규제근거_출처.md , 규제필터_면책_및_심의경로.md .

11-2. 사용 모델 · 오픈소스 · 산출 문서

구분	항목	용도
LLM	OpenAI GPT-5.4-mini (API)	문구 생성, 규제 2차 판정, 시각 프롬프트 변환
이미지 생성	Stable Diffusion 1.5 / SDXL	draft 단계(768px 시안 3장) / refine 단계(고품질 렌더링)
배경 제거	rembg (U ² -Net)	제품 마스크 추출
추론	Hugging Face <code>diffusers</code> , <code>transformers</code>	파이프라인 구성, CLIP tokenizer
이미지 처리	Pillow (PIL)	문구 합성, 워터마크, 캔버스 조립
폰트 검증	fontTools	글리프 커버리지 실측 (18종)
백엔드 · 프론트	FastAPI · Uvicorn / React 19 · Vite	모델 API 서버 / 웹 클라이언트
테스트	pytest, Playwright, oxlint	단위·통합·E2E·정적 검사
인프라	GCP Compute Engine (g2-standard-4 + NVIDIA L4), systemd	배포
문서	<code>docs/UIUX_스펙정리.md</code>	화면 흐름·API 계약의 단일 진실 원천
문서	<code>docs/AI_생성물_고지_표준안.md</code>	AI기본법 제31조 대응 3지점 구현 기준
문서	<code>docs/설계_LLM검증레이어.md</code> · <code>docs/실험결과_규제필터.md</code>	하이브리드 라우팅 설계 / 골드셋·A/B 측정 결과
문서	<code>copy_model/README.md</code> · <code>poster_model/README.md</code>	API 명세, 규제 룰 표, 파이프라인 구조
기록	GitHub Discussions	협업 일지 82건 (Daily log + Total daily log)

11-3. 용어

용어	설명
<code>block / warn / safe</code>	규제 판정 3단계 — 사용 불가 / 맥락 확인 필요 / 문제 없음
골드셋 · train/holdout/redteam	사람이 라벨링한 검증 데이터셋 — 튜닝 대상 / 미노출(실제 성능 대표) / 의미회피 스트레스셋
라벨 정확일치	예측 등급이 정답 등급과 그대로 일치하는 비율. 이진 탐지와 달리 등급 오류를 잡는다
에스컬레이션	룰 1차가 경계 케이스일 때만 LLM 2차로 넘기는 것
단조 신뢰 정책	최종 등급이 룰 1차 등급보다 낮아지지 않도록 고정
draft / refine	가벼운 모델의 초안 시안 / 선택 시안의 고품질 렌더링
footprint	bbox가 아니라 그림자까지 포함한 배치 계산 영역
canvas bridge	제품 추출용 정사각 소스와 최종 출력 캔버스를 잇는 변환 계층
fail-closed	잘못된 값이 들어오면 보정하지 않고 거부
Claim-Lock	문구의 사실 근거를 사용자 입력으로 묶어 LLM 날조를 막는 설계
prompt budget	CLIP 토큰 예산(72/77) 안에 프롬프트를 맞추는 처리
<code>spec</code>	챗봇이 수집한 사용자 입력 상태. stateless 구조에서 프론트가 들고 다닌다
<code>subject_kind</code>	<code>product / service</code> 구분 축. 서비스형에 제품 전용 프롬프트 접미사가 붙지 않게 함

11-4. 부록 — 본문 항목별 Pull Request 대조표

본문은 읽기 흐름을 위해 Pull Request 번호를 넣지 않았음. 코드 이력을 함께 확인해야 할 때 아래 표를 참조함. 저장소는 `github.com/Final-Project-team5/Final-Project-team5` 이며, `https://github.com/Final-Project-team5/Final-Project-team5/pull/<번호>` 형식으로 접근할 수 있음.

2026-08-29 확인 기준 PR 번호는 #152까지 생성됨. 번호가 연속적이지 않아 번호 자체를 전체 건수로 보지 않았고, **#149는 병합 완료, #148과 #152는 확인 시점에 열려 있는 문서 PR**이었음. 따라서 본문 구현 상태는 병합·실측된 코드와 주간 보고서를 기준으로 서술하고, 열린 문서 PR은 대조표에 상태를 명시함.

3장 · 서비스 소개

본문 항목	PR
챗봇 순차 슬롯필링 · 특정 항목만 묻는 옵션	#7 · #9
용도 슬롯 추가 → 비율 자동 매핑 · 범위 밖 값 방어	#36 · #51
제품형 / 서비스형 0단계 분기	#62 · #79
제품 Vision 인식과 사용자 확인 절차	#70 · #78 · #88
서비스업 흐름 정식 지원 (5단계 확정)	#76 · #77
조건부 후속질문 · 자유입력 규제 검사 · 출처 태깅	#100 · #103 · #104
화면 7종 구현 및 흐름 개편	#8 · #18 · #25 · #38 · #45 · #53
배경 방식 선택 화면 신설	#94

4장 · 시스템 설계와 기술 선정

본문 항목	PR
문구 모델 API 초안 (생성 · 슬롯필링 · 규제 검증)	#7
한국어 요청 → 영어 시각 프롬프트 변환	#127
이미지 생성 2단계 파이프라인	#16
비정사각 비율 · 문구 전용 재합성 경로	#44
상태코드 규약 정리 (502 → 400, 예외 재전파)	#87 · #101
폰트 18종 확충 및 글리프 검증	#27

5-1 · 문구 모델 · 규제 검증 · 챗봇

본문 항목	PR
규제 룰 확충 · CORS 설정	#10
시안별 safe 필드 · 다양성 검사	#17
규제 위반율 평가 파이프라인	#19
골드셋 회귀 테스트 · 허용표현 화이트리스트	#31
지표 정직화 — 라벨 정확일치율 · 경계 케이스 편입	#37
LLM 2차 검증 레이어 (하이브리드 라우팅 · 레드팀)	#52
재검증 엔드포인트 연결 · 신뢰 정책 확정	#68
서비스업(학원 / 체육관·도장) 규제 룰	#61
커버리지 갭 보강 (의료행위 오인 · 환불 측)	#99
goods 의약품·의료기기 오인 block 룰 신설	#102
신규 룰 골드셋 편입 · 회귀 잠금	#111
"100%" 표현 오탐 방어	#110
<code>.env</code> 로드 · <code>.gitignore</code> 추가	#63
배경 레퍼런스 Vision 분리 · 파싱 방어	#80 · #86
매력 문구 A/B 하네스 복원 및 실측	#119
호출 타임아웃 · 재시도 백오프	#120
서비스형 시각 프롬프트에서 업증명 제거	#138
문서 최신화 · 규제 룰 표 실측 정정	#81 · #129

5-2 · 포스터 생성 모델

본문 항목	PR
포스터 생성 파이프라인 · 제품 보존 구조	#16
비정사각 지원 · 문구 재합성 · 폰트 적용	#44
프론트 개발 서버 CORS 허용	#57
고품질화 첫 요청 지연 최적화	#69
Design Planner Core	#95
타이포그래피 톤 프리셋 API 연결	#115
동시성 직렬화 · 프롬프트 예산 (77토큰 대응)	#116
제품형/서비스형 구분 파라미터 배선	#121
고품질화 단계 메모리 부족 해결	#128
서비스형 배경 품질 — 해상도·강도 분리	#134
인물 배제 → 서비스형 전용 분리	#140 → #142
포스터 README · API 문서 최신화	#148

5-3 · 프론트엔드 · 백엔드 연동

본문 항목	PR
홈 · 화면 A~E 구현, 흐름 개편, 로딩·에러 UI	#8 · #18 · #25 · #38 · #45
브랜드 리뉴얼 (마스코트 · 사이드바)	#53
문구 모델 실행API 연동 · 포스터 어댑터 준비	#60
챗봇 분기 및 이미지 입력 흐름 개편	#79
배경 Vision 연동 · 제품 이미지 단계 개선	#88
배경 선택 UI	#94
챗봇 후속질문 연동 · 레이아웃 개선	#104
mock 결과에 편집 문구 합성	#109
고품질화 단계 제품 분류 전달 · 폰트 미리보기	#133
개발 서버 폰트 로드 · 문구 외곽선 정합	#135
3:1 배너 문구 크기 보정	#143
미리보기 줄바꿈 정합 · 배너 프리셋	#147
최종 로딩 연출 · 실제 예시 이미지 · UI/UX 스펙 최신화	#149

문서 최신화

본문 항목	PR
최상위 README 최신 구조 반영	#152

6장 · 개발 과정과 의사결정

본문 항목	PR
협업 일지 게시판 템플릿 도입	#2
폰트 검증 문서화	#27

8장 · 문제 해결 과정

본문 이슈	발견 → 해결
A/B 실험이 규칙 유무와 무관하게 위반율 0%	#19 → #31
규제 필터 성능지표가 실제보다 높게 보고됨	#31 → #37
커버리지를 케이스 수 / 룰 수로만 측정하던 한계	#61 → #99 · #102
후속질문 트리거의 한 글자 토큰 오탐	#100 → #103
프론트 연동 중 간헐적 500 오류	#116
프롬프트가 토큰 한도를 초과해 뒷부분이 잘림	#116
고품질화 반복 시 프로세스 강제 종료	#128
미리보기와 최종 출력 불일치 5건	#133 · #135 · #143 · #147

코드 리뷰가 설계를 바꾼 사례

본문 6-7절과 10-2절에서 언급한 교차 리뷰의 근거임.

리뷰	지적	반영 결과
#31 리뷰 (PM)	이진화 과정에서 등급 구분이 사라져 등급 오류가 지표에 잡히지 않음	#37 — 1차 지표를 3-클래스 라벨 정확일치율로 교체
#61 리뷰 (PM)	룰당 케이스 밀도가 1.4건이라 커버리지 범위를 알 수 없음	#99 · #102 — 커버리지 매트릭스 도입 후 겹 3건 해소
#68 리뷰 (PM · 프론트)	등급 하향 가드가 형식 검사라 사실상 무력	하향 전면 차단(단조 신뢰 정책) + 통합 테스트 추가
#70 리뷰 (프론트)	Vision 결과 자동 확정은 오인식 시 연쇄 오류	#78 — 확인 절차와 전용 확정 엔드포인트 신설
#87 리뷰 (프론트)	핸들러 예외가 502로 덮여 원인 파악이 어려움	#101 — 예외 재전파로 상태코드 정리
#100 리뷰 (PM)	사실값을 코드로 검증하려면 값마다 출처가 필요	#103 — 수집값 출처 태깅 필드 추가

리뷰 참여 (본인 작성 PR 제외, 한 PR에 여러 의견을 남겨도 1건으로 집계) — 김소원 30건 / 정진우 29건 / 황지우 10건 / 김도혁 6건.